

RECAP: Reward-Calibrated Preference Optimization for Low-Resource Machine Translation

Shashwat Bhardwaj Vaibhav Sagar
Indian Institute of Technology Delhi

Abstract

RECAP is a framework for applying preference optimization to low-resource¹ neural machine translation (NMT), where parallel data and human preference labels are scarce. Its primary goal is to improve held-out translation metrics over the corresponding supervised fine-tuning (SFT) baseline. For each translation direction, RECAP starts from an existing dataset of SFT-finetuned NLLB, mT5, Qwen, and Llama outputs, constructs reliable pseudo-preferences² with a direction-calibrated³ reward based on BLEU, ChrF++, COMET, repetition, and length mismatch, and trains DPO from the corresponding existing mT5 SFT checkpoint. The policy can then be refined with Group Relative Policy Optimization (GRPO), while Proximal Policy Optimization (PPO) serves as an RL baseline. Success requires improvements over SFT on held-out BLEU, ChrF++, and COMET without increasing repetition or severe length mismatch.

1 Motivation

Low-resource NMT is difficult because the available parallel corpus is small. SFT can learn from this corpus, but it does not tell the model which of two generated translations is better. In high-resource settings, this signal can come from human preference data or strong learned evaluators. These resources are often unavailable for low-resource languages. The practical goal is therefore to use preference optimization or RL to improve translation quality and held-out metrics beyond the initial SFT model, despite this lack of direct preference supervision.

We study six independent translation directions between Hindi and Bhili, Gondi, and Mundari:

$$\begin{aligned}\mathcal{D} = \{ & \text{Hindi} \rightarrow \text{Bhili}, \text{Bhili} \rightarrow \text{Hindi}, \\ & \text{Hindi} \rightarrow \text{Gondi}, \text{Gondi} \rightarrow \text{Hindi}, \\ & \text{Hindi} \rightarrow \text{Mundari}, \text{Mundari} \rightarrow \text{Hindi} \}.\end{aligned}\tag{1}$$

Each direction is a separate task. We do not share training examples, model parameters, rewards, or preference pairs across directions. For direction d , the parallel corpus is

¹*Low-resource* does not have one universal numerical cutoff. It refers to a language or translation direction with limited usable digital and parallel data, NLP tools, and evaluation resources. The literature treats resource level as a continuum shaped by data, linguistic artifacts, technological infrastructure, and human/community resources; it is therefore not determined by speaker population alone [4, 6]. In this paper, *extremely low-resource* is an operational term for directions with only a small parallel corpus and sparse reliable language-specific tools or benchmarks.

²A *pseudo-preference* is an automatically created “better translation versus worse translation” label. It is inferred from the reward and the available reference translation, rather than collected from a human who directly compares the two outputs.

³*Calibration* means adjusting and interpreting reward scores separately for each translation direction, using examples from that direction, so that a score has a meaningful scale there. In simple terms, it avoids treating the same raw metric score as equally good for languages and directions with different difficulty, scripts, or metric behaviour.

$$\mathcal{T}_d = \{(x_i^{(d)}, y_i^{*(d)})\}_{i=1}^{N_d}, \quad (2)$$

where $x_i^{(d)}$ is a source sentence and $y_i^{*(d)}$ is its reference translation. RECAP asks: *how can we construct reliable translation preferences when human labels and strong target-language evaluators are not available?*

1.1 Problem Setting and Assumptions

The framework assumes that a small parallel corpus is available for each direction and that four SFT-finetuned candidate outputs—NLLB, mT5, Qwen, and Llama—have already been generated and stored for each source. It does *not* retrain these four generators merely to construct preferences. It also assumes access to the direction-specific mT5 SFT checkpoint for DPO, GRPO, and PPO; stored text outputs alone are sufficient for offline preference mining, but not for updating a policy. RECAP does *not* assume a large set of human comparisons such as “translation A is better than translation B.” Instead, it creates pseudo-preference data from the stored outputs and a carefully designed reward.

For every source sentence, the reference translation is used only to construct and validate the reward during training. At inference time, the final model translates a source sentence without seeing its reference. This distinction is important: the reference is a training-time supervision signal, not an input to the deployed translation model.

The six directions may differ in corpus size, script, morphology, and available pretrained models. For this reason, pooling all examples into one generic preference dataset can hide direction-specific behavior. RECAP instead builds a separate data pipeline for every $d \in \mathcal{D}$.

2 Literature Survey

RECAP builds on several established lines of work: sequence-level optimization for machine translation, preference optimization and reinforcement learning for language models, MT-specific preference learning, multilingual low-resource translation, and machine translation evaluation.

2.1 Sequence-Level Optimization for Machine Translation

Before preference optimization became common, machine translation research already explored objectives that go beyond token-level maximum likelihood training. Minimum Risk Training (MRT) directly minimizes an expected sentence-level loss over candidate translations and showed that sequence-level objectives can improve NMT quality [11]. Reinforcement-learning studies have also examined how policy-gradient methods can optimize MT models against sequence-level rewards, while identifying sensitivity to reward design and training stability [13]. RECAP shares the goal of improving translation beyond SFT, but uses pairwise pseudo-preferences and DPO as its main optimization route rather than optimizing only a single sequence-level metric.

2.2 Preference Optimization and Reinforcement Learning

Direct Preference Optimization (DPO) was introduced as a way to optimize a policy directly from preference pairs without separately fitting a reward model and running the full RLHF pipeline [7]. PPO remains an important clipped policy-gradient baseline for reinforcement learning [9]. GRPO uses relative rewards within a group of samples and was introduced as a memory-efficient alternative to critic-based PPO in the DeepSeekMath work [10]. In RECAP, DPO is the primary optimizer

because the central question is whether reliable pseudo-preferences can improve translation over SFT. GRPO and PPO are secondary tests of whether the same reward foundation supports on-policy⁴ refinement.

2.3 Preference Optimization for Machine Translation

Several recent studies show that preference optimization can improve MT, so RECAP should not claim novelty merely from applying DPO to translation. Yang et al. use DPO to distill the gains of Minimum Bayes Risk decoding into multilingual translation models [14]. Uhlig et al. introduce Direct Quality Optimization, which uses a pre-trained translation quality-estimation model as a proxy for human preferences in multilingual NMT [12]. Cui et al. propose confidence-reward driven preference optimization for MT and emphasize that DPO depends strongly on preference-data quality [1]. These studies are closely related to RECAP.

2.3.1 CRPO and RECAP

CRPO is short for Confidence–Reward Driven Preference Optimization. It is an offline⁵ method for selecting DPO data. For each source sentence, it samples several translations from one reference policy and scores them with a point-wise reward model. It then selects a preferred–rejected pair using a CR-Score that combines the reward difference with the reference policy’s relative log-likelihood of the two translations. Intuitively, CRPO retains a pair when it has a useful quality gap and the policy still fails to prefer the better translation. Its “confidence” is therefore *model-likelihood confidence*: it measures what the current reference policy is likely to generate and prioritizes difficult, high-learning-potential pairs [1].

RECAP uses a different notion, called *reward-margin label confidence*. For RECAP, a pair is reliable when its direction-calibrated multi-aspect reward has a sufficiently large better–worse margin; the margin estimates confidence in the *pseudo-label*, not the likelihood assigned by the SFT policy. RECAP also begins with heterogeneous candidate translations from NLLB, mT5, Qwen, and Llama rather than samples from one reference policy. Under the stated zero-human-label setting, its reward combines fixed BLEU, ChrF++, and COMET signals with repetition and length mismatch; it does not claim that these automatic signals verify fluency, language identity, hallucination, or human preference. Finally, RECAP studies whether this direction-isolated construction works across six Hindi↔Bhili/Gondi/Mundari translation directions and optionally tests later online GRPO refinement.

Thus, CRPO is a particularly important baseline, not a method that RECAP replaces by definition. A fair comparison should distinguish reward-margin label confidence from CRPO’s model-likelihood confidence and test whether RECAP’s heterogeneous candidates, calibration, error-aware reward, and disagreement analysis provide benefits beyond a CRPO-style reward-plus-likelihood selection rule.

2.4 How RECAP Differs from Existing Approaches

Table 1 makes the intended distinction explicit. It is a comparison of the representative approaches discussed above, not a claim that no earlier paper has used any one RECAP component. RECAP’s

⁴ *On-policy* means that the model is trained using fresh translations generated by the current version of that same model. After an update, the changed model produces new translations for the next update.

⁵ *Offline* training uses a fixed dataset created before the current training run; the model does not need to generate new outputs after every update. In reinforcement-learning language, learning from outputs made by another or an older policy is often called *off-policy*.

contribution is the *combination* of heterogeneous candidate translations, direction-calibrated and error-aware scoring, confidence-filtered pseudo-preferences, and evaluation across six extremely low-resource directions.

Thus, RECAP is differentiated by its intended setting and empirical question: it starts from a direction-specific dataset containing several heterogeneous candidate translations for the same source sentence, constructs a multi-aspect and direction-calibrated pseudo-preference signal, filters uncertain pairs using reward margins, and tests whether this procedure improves the SFT baseline across six extremely low-resource directions. This distinction must be demonstrated experimentally through ablations, rather than assumed from the framework description.

2.5 Low-Resource Multilingual Translation and Evaluation

The Bhili, Gondi, and Mundari setting is itself an important part of the problem. Kamau studies adaptation of NLLB-200 and mBART for Bhili, Gondi, Mundari, and Santali, using data-centric validation and conservative handling of catastrophic generation errors [5]. This provides direct evidence that these directions present low-resource challenges where robustness and failure detection matter.

RECAP also relies on automatic scores when constructing and evaluating preferences, so metric reliability is critical. COMET introduced neural MT evaluation models that aim to predict human translation-quality judgments [8]. However, Falcão et al. show that neural evaluation models can have important language-support limitations in low-resource settings [2]. RECAP therefore fixes and documents one COMET configuration, reports it separately from BLEU and ChrF++, and treats it as an imperfect reference-aware signal rather than a human-quality oracle.

2.6 Research Gap

Prior work establishes that sequence-level optimization, DPO, quality-estimation-based alignment, and confidence-aware MT preference selection can improve translation. The open question addressed by RECAP is narrower: *can heterogeneous candidate outputs and direction-calibrated, error-aware pseudo-preferences provide a reliable preference signal for improving SFT metrics in extremely low-resource translation directions?* The main contribution must therefore be supported by controlled evidence that RECAP preference construction outperforms simpler alternatives, including raw metric ranking, self-sampled candidates, and unfiltered or random preference pairs.

3 Dataset and Candidate Pool

The empirical basis for RECAP is a multi-model candidate translation dataset, rather than a pre-existing preference dataset. For each low-resource language—Bhili, Gondi, and Mundari—we retain two direction-specific files: `hi2tgt.csv` and `tgt2hi.csv`. Each row provides a `source`, a gold reference `gold_truth`, four model translations, and already stored sentence-level BLEU and ChrF++ values for each model translation. A fixed reference-aware COMET score is computed later from the source, candidate, and reference for every candidate.

For source x_i , gold reference y_i^* , and candidate model $m \in \{\text{NLLB}, \text{mT5}, \text{Qwen}, \text{Llama}\}$, the row has the form

$$\left(x_i, y_i^*, \left\{ y_i^{(m)}, b_i^{(m)}, h_i^{(m)}, c_i^{(m)} \right\}_{m=1}^4 \right), \quad (3)$$

where $y_i^{(m)}$ is the candidate text, $b_i^{(m)}$ is the stored BLEU value, $h_i^{(m)}$ is the stored ChrF++ value, and $c_i^{(m)}$ is the COMET score computed with the fixed configuration for candidate m . In the provided CSV samples, BLEU and ChrF++ are reported on a 0–100 scale: for example, an exact candidate–reference match has values 100 and 100. RECAP first converts stored BLEU and ChrF++ to $[0, 1]$ and then standardizes all three quality metrics within the training candidates of a direction.

$$\hat{b}_i^{(m)} = \frac{b_i^{(m)}}{100}, \quad \hat{h}_i^{(m)} = \frac{h_i^{(m)}}{100}. \quad (4)$$

The stored metric columns are used only after verifying and documenting the metric implementation, tokenization, and sentence-BLEU smoothing that produced them. If those settings are unavailable or inconsistent across files, RECAP recomputes the metrics from the candidate and `gold_truth` fields with one fixed configuration. The COMET checkpoint, score direction, source/reference inputs, and language coverage must likewise be fixed before preference mining. The following examples illustrate the variation available for preference mining. They are model hypotheses and metric values, not human preference labels.

3.1 CSV Schema and Real Sample Rows

Table 2 maps the retained file names to the actual candidate and metric columns. Here, `tgt` is Bhili, Gondi, or Mundari, and the output-language prefix changes accordingly (for example, `Bhili_nllb` or `Hindi_nllb`).

The retained rows below are real samples supplied from the Hindi-centred CSV files. They show why RECAP stores both the candidate text and its sentence-level metrics: model rankings can differ by source, several models can tie exactly, and a repetitive output can receive high reference scores when the reference itself repeats material.

Mundari: Hindi $\rightarrow$ Mundari (`hi2tgt.csv`). **Source:** क्या वाराणसी ज़िले में आयु वर्ग 15-17 वर्ष के लिए कोई टीकाकरण केंद्र उपलब्ध है?

Gold reference: चना: वाराणसी जिला रे आयु वर्ग चना: वाराणसी जिला रे आयु वर्ग 15-17 सिरमा मेनते जान टीकाकरण केंद्र मेनअ: ? सिरमा मेनते जान टीकाकरण केंद्र मेनअ: ?

Model	Candidate	BLEU	ChrF++
NLLB	चिनअ: वाराणसी जिला रे उमर वर्ग 15-17 सिरमा मेनते जान टीकाकरण केंद्र मेनअ: ?	33.0495	49.6601
mT5	चिनअ: वाराणसी जिला रे उमर वर्ग चिनअ: वाराणसी जिला रे उमर वर्ग 15-17 सिरमा मेनते जान टीकाकरण केंद्र मेनअ: ? सिरमा मेनते जान टीकाकरण केंद्र मेनअ: ?	70.6167	80.6730
Qwen	चिनअ: वाराणसी जिला रे आयु वर्ग 15-17 सिरमा मेनते जान टीकाकरण केंद्र मेनअ: ?	41.2953	55.0619
Llama	चिनअ: वाराणसी जिला रे उमर वर्ग चिनअ: वाराणसी जिला रे उमर वर्ग 15-17 सिरमा मेनते जान टीकाकरण केंद्र मेनअ: ? सिरमा मेनते जान टीकाकरण केंद्र मेनअ: ?	70.6167	80.6730

This row demonstrates why the repetition penalty is included even when BLEU and ChrF++ are high: the reference itself contains repeated material, and the mT5/Llama candidates repeat it closely. The penalty records observable repeated n -grams independently of the stored reference metrics. This does not prove that the repetition is linguistically wrong; it makes the model’s

repetition behavior explicit and lets the ablation test whether the penalty improves downstream held-out results.

The following additional candidate texts illustrate the same four-model structure across the six retained directions. They are included to show the range of output wording; the stored per-row BLEU and ChrF++ columns described above are the values used for the automatic reference-fidelity component.

3.2 Bhili Examples

Hindi → Bhili. Source: संगीत कार्यक्रम आमतौर पर एक वर्नम या आह्वान के साथ शुरू होता है जो उद्घाटन के रूप में कार्य करता है।

NLLB: संगीत कार्यक्रम आमतौर पर एक वर्नम या आह्वान नी हाते शुरू थाय से जो उद्घाटन ना रूप मां कार्य करे से ।

mT5: संगीत कार्यक्रम आमतौर पर एक वर्नम या आह्वान नी हाते शुरू थाय से जी उद्घाटन ना रूप मां कार्य करे से ।

Qwen: संगीत कार्यक्रम आमतौर पर एक वर्नम या आह्वान नी हाते शुरू थाय से जो उद्घाटन ना रूप मां कार्य करे से।

Llama: गीत ना कार्यक्रम खासतौर पर एक वर्नम अन बोलावा नी हाते शुरू थाय से त्यो उद्घाटन ना रूप मां काम करे से ।

Bhili → Hindi. Source: गीत एक खास मेला मां तिना मेलानी होत शुरू थाय से जो तिना मेला में तिना शुरू थावा वाला काम होते काम करे से ।

NLLB: गीत एक विशेष मेले में उन मेहमानों द्वारा शुरू होता है जो उस मेले में उनके प्रारंभिक कार्य के रूप में काम करते हैं।

mT5: संगीत एक विशिष्ट संगोष्ठी के साथ शुरू होता है जो संगोष्ठी के प्रारंभिक कार्यों के साथ काम करता है।

Qwen: संगीत एक महत्वपूर्ण समूह में उस समूह की शुरुआत होती है जो उस समूह में उस शुरुआती कार्य होता है।

Llama: संगीत कार्यक्रम आमतौर पर उन्हीं शैली के साथ शुरू होते हैं जो शेष कार्यक्रम की शैली होती है।

3.3 Gondi Examples

Hindi → Gondi. Source: इस वर्ष हम चंपारण सत्याग्रह और साबरमती आश्रम की शताब्दी भी मना रहे हैं।

NLLB: इद वरसा माट चंपारण सत्याग्रह अन साबरमती आश्रम ता शताब्दी गला माने मायतोरट।

mT5: इद वरसा माट चंपारण सत्याग्रह अन साबरमती आश्रम ता शताब्दी गला मानेह किययतोरट।

Qwen: इद वरसा माट चंपारण सत्याग्रह अन साबरमती आश्रम ता शताब्दी गला माने मायतोरट।

Llama: इद वरसा माट चंपारण सत्याग्रह अन साबरमती आश्रम त शताब्दी गला मानेह किययतोरम।

Gondi → Hindi. Source: इद वरसा माट चंपारण सत्याग्रह अन साबरमती आश्रम त शताब्दी गला मानेह कियय तोरट।

NLLB: इस वर्ष हम चंपारण सत्याग्रह और साबरमती आश्रम की शताब्दी भी मना रहे हैं।

mT5: इस वर्ष हम चंपारण सत्याग्रह और साबरमती आश्रम की शताब्दी भी मना रहे हैं।

Qwen: इस वर्ष हमने चंपारण, सत्याग्रह और साबरमती आश्रम की शताब्दी भी मनाई।

Llama: इस वर्ष हम चंपारण सत्याग्रह और साबरमती आश्रम की शताब्दी भी मना रहे हैं।

3.4 Mundari Examples

Hindi → Mundari. Source: लेकिन युवक युवतियों का नृत्य एक सप्ताह तक चलते रहता है।

NLLB: मेनदो दंगड़ा दंगिड़ि कोअ: सुसुन मोद् हपता जकेद् सेसेन तना।

mT5: मेनदो धंगड़ा धंगिड़िकोअ: सुसुन मोद् पीटि लेका सेसेनाकनगे तइना।

Qwen: मेनदो दंगड़ा-दंगिड़ि कोअ: सुसुन मिद् अठाउरि जकेद् सेसेन तना।

Llama: मेनदो दंगड़ा दंगिड़िकोअ: सुसुन अठाउरि जकेद सेसेना।

Mundari → Hindi. Source: मेनदो धंगड़ा धंगिड़िकोअः सुसुन दो अठाउरि दिन जकेद् तइन तना ।

NLLB: लेकिन युवक युवतियों का नाच तो आठ दिन तक रहता है।

mT5: लेकिन युवक युवतियों का नृत्य एक सप्ताह तक रहता है।

Qwen: लेकिन युवक युवतियों का नृत्य तो सप्ताह तक रहता है।

Llama: लेकिन युवक युवतियों का नृत्य एक सप्ताह तक रहता है।

These examples demonstrate why per-sentence ranking is necessary. Global model scores provide a useful prior, but they cannot identify which candidate has the strongest reference fidelity or the fewest observable repetition and length problems for a particular row. RECAP uses its reference-guided reward to make that limited comparison at the row level.

4 Candidate Generator Performance

Tables 3–5 report the validation and test results of the four SFT-finetuned systems whose outputs are already stored in the candidate dataset before RECAP. Each score cell is written as BLEU / ChrF++. Placing this analysis after the dataset description makes clear why several stored output columns are retained: the candidate pool is heterogeneous in quality, and the strongest model can change across directions.

Common SFT backbone selection. The common SFT backbone for the DPO, GRPO, and PPO comparisons is **mT5**. This choice uses only the validation columns of Tables 3–5: for each model, we macro-average BLEU and ChrF++ across the six Hindi-centred directions, giving every direction equal weight. mT5 has the strongest validation average (41.97 BLEU and 61.23 ChrF++), ahead of Llama (39.75 BLEU and 59.31 ChrF++), NLLB (34.47 BLEU and 55.79 ChrF++), and Qwen (24.41 BLEU and 45.85 ChrF++). It is therefore the strongest common starting point across the full study, even though Llama is best for the individual Hindi → Bhili direction. The test columns are not used for this decision.

The results nevertheless show useful direction-specific variation. On the Bhili test sets, Llama is strongest for Hindi → Bhili, while mT5 is strongest for Bhili → Hindi. On the retained Gondi and Mundari directions, mT5 has the highest reported test BLEU and ChrF++ among the four stored candidate-output systems. Qwen is consistently weaker in this reference summary, although it may still supply useful sentence-level alternatives. All four stored outputs remain in the candidate pool; selecting the existing mT5 checkpoint as the trainable backbone does not discard NLLB, Qwen, or Llama candidates.

The score gaps also motivate a necessary control: if RECAP always pairs a globally strong model with Qwen, the method may become simple strong-teacher distillation. The experiments should therefore report the generating-model composition of chosen and rejected candidates, and compare best-versus-worst pairs with near-neighbor and self-sampling alternatives that are matched for pair count and reward-margin distribution.

5 Research Objective and Contributions

We want to improve machine translation for Bhili, Gondi, and Mundari, where training data and human feedback are limited. For each source sentence, we already have several translations from different models. These candidates differ in their overlap with the available reference and can also show observable problems such as repetition or unusually short or long output.

Our goal is to use these different translations to learn which output is better for each individual sentence. We then use the better–worse comparisons to train a translation model with DPO or,

optionally, further RL refinement. Instead of assuming that one model is always best, we choose the better translation separately for every source sentence.

In simple terms, RECAP tries to turn existing model outputs into useful training feedback. Its main test is whether this feedback lets the final model beat its own SFT baseline on held-out BLEU, ChrF++, and COMET without increasing repetition or severe length mismatch. We evaluate every translation direction separately because Hindi, Bhili, Gondi, and Mundari have different data sizes and metric scales.

5.1 Research Questions and Hypotheses

The primary hypothesis is that RECAP-generated preferences enable DPO, and optionally later RL refinement, to improve held-out translation metrics over SFT. We test whether:

- heterogeneous models provide better candidates than repeated samples from one model;
- direction calibration improves preference reliability over raw metrics;
- confidence filtering reduces noisy labels;
- disagreement identifies informative examples; and
- DPO provides a stronger starting point for GRPO than SFT alone.

The contribution is not DPO or GRPO by themselves. It is a practical pipeline for constructing preference data when only parallel references and heterogeneous model outputs are available:

$$\begin{array}{l}
 \text{heterogeneous candidates + direction calibration} \\
 + \text{ observable reward + confidence + disagreement} \\
 \implies \text{reliable preferences} \\
 \implies \text{DPO} \implies \text{optional GRPO.}
 \end{array} \tag{5}$$

The central claim is therefore

$$\boxed{\text{Preference Quality} > \text{Preference Quantity.}} \tag{6}$$

6 RECAP Method

RECAP means **RE**ward-**C**alibrated **A**lignment for **P**reference Optimization. Reliable preference construction is the means; improving translation quality and metrics beyond SFT is the end goal. The framework has five stages:

6.1 Method Overview and Pipeline

The title uses the shorter phrase “Reward-Calibrated Preference Optimization,” while the full name makes the middle step explicit: **RE**ward-**C**alibrated **A**lignment for **P**reference Optimization. RECAP is not another name for DPO alone. DPO is one optimizer inside the framework. The name instead describes how the complete algorithm converts several imperfect translations into training feedback that moves an SFT model toward better translations.

Reward (R)

A reward is the score assigned to each candidate translation. It answers, “Which

candidate is preferable under the available evidence?” RECAP combines a reference-aware quality component (BLEU, ChrF++, and COMET) with repetition and length-mismatch penalties. It does not claim that the score detects meaning, fluency, language identity, or unsupported content without a validated evaluator.

Calibrated (C)

Raw component values mean different things in different directions. For example, the same BLEU or repetition value may be normal for Hindi $\rightarrow$ Gondi but unusual for Gondi $\rightarrow$ Hindi. Calibration normalizes and weights the three reward components separately for each direction. It answers, “What counts as relatively high or low *for this translation direction*?”

Alignment (A)

Alignment means changing the SFT policy so that it assigns more probability to candidates with higher calibrated reference fidelity and fewer observable repetition or length problems. It does *not* mean general-purpose value alignment, verified semantic fidelity, or a claim that the model replaces human cultural judgment.

Preference (P)

A preference is a better–worse comparison between two translations of the same source. RECAP creates these pseudo-preferences from the calibrated reward: the higher-scoring candidate is the chosen output and the lower-scoring candidate is the rejected output. Reward-margin label confidence and candidate disagreement determine whether that comparison is reliable enough to retain.

Optimization (O)

Optimization is the parameter update that learns from the preferences. Offline DPO is the main update: it learns from the fixed chosen–rejected pairs. Optional online GRPO can further refine the DPO policy using fresh groups of translations, while PPO is a controlled RL baseline. Optimization is the final learning step, not the source of the preference labels.

The parts of the algorithm not named directly by a letter are the safeguards that make RECAP practical. Heterogeneous stored candidate outputs supply different possible translations; direction isolation prevents an easier direction from setting the standards for a harder one; reward-margin filtering removes uncertain labels; and disagreement sampling focuses training on examples where candidate quality differs. Together, the full logic is

$$\begin{array}{l}
 \text{heterogeneous candidates} \\
 \implies \text{reward} \implies \text{direction calibration} \\
 \implies \text{reliable preferences} \implies \text{translation alignment} \\
 \implies \text{DPO} \implies \text{optional GRPO.}
 \end{array} \tag{7}$$

Thus, the name RECAP emphasizes the central scientific claim of the framework: carefully *reward-calibrated* preference construction should make preference optimization a reliable way to improve low-resource translation beyond SFT.

1. Start from the stored outputs of independent SFT-finetuned models for each translation direction.

2. Use the four stored candidate translations from NLLB, mT5, Qwen, and Llama.
3. Score every candidate with a direction-calibrated, multi-aspect reward.
4. Keep high-confidence preference pairs, with emphasis on useful model disagreement.
5. Train with DPO and optionally refine the policy online with GRPO.

In short,

$$\boxed{\text{stored SFT outputs} \rightarrow \text{reward calibration} \rightarrow \text{preference mining} \rightarrow \text{DPO} \rightarrow \text{GRPO}.} \quad (8)$$

PPO is evaluated separately as a controlled RL baseline.

6.2 Direction-Isolated Training

For every direction d , RECAP trains a separate policy $\pi_{\theta_d}(y | x)$. If $d \neq d'$, then the policies have no shared parameters:

$$\theta_d \cap \theta_{d'} = \emptyset. \quad (9)$$

The reward is also calibrated separately for each direction. In general,

$$R_d^{\text{RECAP}}(x, y, y^*) \neq R_{d'}^{\text{RECAP}}(x, y, y^*). \quad (10)$$

This is important because the data size, error patterns, and metric scales may differ across directions. The full problem is therefore six independent optimization problems:

$$\min_{\theta_d} \mathcal{L}_d, \quad d \in \mathcal{D}. \quad (11)$$

Direction isolation applies to checkpoints and reporting, but not to the initial model-family choice. mT5 is selected once as the common trainable backbone because it has the highest macro-average *validation* BLEU and ChrF++ across the six directions in Tables 3–5. For every direction, RECAP uses the existing direction-specific mT5 SFT checkpoint that produced the stored mT5 output; it does not create a shared checkpoint or retrain the four candidate generators during preference mining. Calibration statistics, reward weights, confidence thresholds, and held-out evaluation sets remain separate. This retains a strong common starting point while preventing an easier direction from dominating another direction’s reward calibration or checkpoint selection.

6.3 Heterogeneous Candidate Collection

For a source sentence x , the stored outputs of independently SFT-finetuned systems—NLLB, mT5, Qwen, and Llama—form the candidate set:

$$\mathcal{Y}(x) = \{y^{\text{NLLB}}, y^{\text{mT5}}, y^{\text{Qwen}}, y^{\text{Llama}}\}. \quad (12)$$

Using different models is useful because their mistakes are not identical. Their disagreement creates candidate translations with different strengths and weaknesses. If candidate rewards vary widely, we define disagreement as

$$D(x) = \text{Var}(R(y_1), \dots, R(y_K)), \quad (13)$$

where K is the number of candidate models. High disagreement may indicate a difficult but informative training example.

6.3.1 Candidate Collection Protocol

When checkpoint access permits regeneration, RECAP can run a candidate-diversity feasibility diagnostic on a fixed 5K-source subset of the training split. For a single SFT model such as NLLB, it compares greedy decoding (beam = 2), beam search with four returned hypotheses, and stochastic sampling with $K = 4$ outputs. The diagnostic reports unique-output ratio, pairwise similarity, sentence-level BLEU, ChrF++, COMET, quality spread, and oracle improvement⁶. This optional diagnostic does not retrain a model or train GRPO; it only tests whether alternative outputs contain enough variation for a meaningful relative-reward signal. The main RECAP preference dataset remains the supplied four-model output dataset.

Candidate diversity must be real rather than an artifact of inconsistent decoding. For each source sentence, every candidate model should use a fixed decoding policy, such as beam search with a stated beam size or sampling with a stated temperature, maximum length, and repetition control. These settings should be logged for every candidate. If one model uses aggressive sampling while another uses deterministic beam search, part of the observed difference may come from decoding rather than model quality.

Before reward computation, candidates should be normalized consistently. Useful checks include Unicode normalization, removal of accidental duplicated whitespace, and preservation of punctuation. The original output should still be stored for auditing. Empty outputs, malformed text, and outputs that copy the source unchanged should be explicitly flagged rather than silently discarded. Script alone is not used to infer the intended language.

For a source sentence x_i , the candidate record can be written as

$$\mathcal{C}_i = \{(m_k, y_{i,k}, q_{i,k})\}_{k=1}^K. \quad (14)$$

Read $\mathcal{C}_i$ as “the complete candidate record for source sentence number i .” The subscript i stays fixed because all candidates translate the same source x_i . The index k identifies one candidate generator, and it runs from 1 to K , where K is the number of generators. In the current dataset, $K = 4$: NLLB, mT5, Qwen, and Llama.

Each parenthesized triple $(m_k, y_{i,k}, q_{i,k})$ contains three pieces of information for one model output:

m_k	The name or identity of candidate model k . For example, $m_1 = \text{NLLB}$, $m_2 = \text{mT5}$, $m_3 = \text{Qwen}$, and $m_4 = \text{Llama}$.
$y_{i,k}$	The translation produced by model m_k for source sentence x_i . The first subscript i says which source row is being translated; the second subscript k says which model produced the output.
$q_{i,k}$	The metadata stored with that translation. This can include decoding settings, output length, repetition flags, generator version, and, after scoring, the individual reward components. Here q means a metadata record; it is <i>not</i> itself the final translation-quality reward $R_{i,k}$.

For one source sentence in this project, the set can be expanded as

⁶ *Oracle improvement* is the gain possible if an ideal selector, after seeing the reference, always chose the best candidate instead of one default output. It measures available candidate diversity, not deployable translation quality.

$$\mathcal{C}_i = \{(\text{NLLB}, y_{i,\text{NLLB}}, q_{i,\text{NLLB}}), \\ (\text{mT5}, y_{i,\text{mT5}}, q_{i,\text{mT5}}), \\ (\text{Qwen}, y_{i,\text{Qwen}}, q_{i,\text{Qwen}}), \\ (\text{Llama}, y_{i,\text{Llama}}, q_{i,\text{Llama}})\}. \quad (15)$$

The curly braces $\{\cdot\}$ mean that all four triples are collected together as one record. Thus, $\mathcal{C}_i$ preserves not only the four translations, but also which model produced each translation and the information needed later to audit its reward and preference label. This record makes the preference-construction process reproducible.

6.3.2 Dataset-to-Preference Transformation

The dataset used by RECAP is not initially a DPO dataset. It is a **multi-model candidate translation dataset**. For each low-resource language—Bhili, Gondi, and Mundari—the retained data are organized into two direction-specific files:

File name	Translation direction
hi2tgt.csv	Hindi $\rightarrow$ target language
tgt2hi.csv	Target language $\rightarrow$ Hindi

Every row contains one source sentence, its gold reference translation, and four already stored outputs from SFT-finetuned NLLB, mT5, Qwen, and Llama systems. For a generic translation direction, a row is represented as

$$(x, y^*, y^{\text{NLLB}}, y^{\text{mT5}}, y^{\text{Qwen}}, y^{\text{Llama}}), \quad (16)$$

where x is the source sentence, y^* is the reference translation, and the remaining terms are already available translations of the same source from four independently SFT-finetuned systems. For Hindi $\rightarrow$ Bhili, these candidate columns correspond to `Bhili_nllb`, `Bhili_mt5`, `Bhili_qwen`, and `Bhili_llama`; for Bhili $\rightarrow$ Hindi, the same role is played by the corresponding Hindi-output columns.

This structure is particularly suitable for RECAP because it provides four alternative hypotheses for exactly the same source and reference. The dataset therefore captures naturally occurring model disagreement rather than artificial comparisons created by changing only a sampling seed. A typical row can be transformed as follows:

$$\begin{aligned} &(x, y^*, y_1, y_2, y_3, y_4) \\ &\xrightarrow{\text{RECAP scoring}} \{R(y_1), R(y_2), R(y_3), R(y_4)\} \\ &\xrightarrow{\text{all pairwise margins and balanced sampling}} \mathcal{P}_i = \{(x, y^+, y^-)\}. \end{aligned} \quad (17)$$

where $\mathcal{P}_i$ is the set of valid DPO pairs derived from source i . A pair contains a preferred candidate y^+ and a rejected candidate y^- whose reward difference exceeds the direction-specific margin. RECAP considers all six unordered pairs among four candidates; it does not restrict the main pipeline to one best–worst pair per source. A later balanced sampler controls pair-type dominance without reversing any preference direction.

The data also contain informative variation. In the Hindi $\rightarrow$ Mundari example, two candidates visibly repeat a phrase even though their stored reference scores are high. Such variation

is valuable because the reward can rank candidates using reference fidelity, repetition, and length mismatch rather than selecting among nearly identical outputs. Without human labels or a validated evaluator, this ranking is a conservative proxy and does not diagnose hidden meaning errors or unsupported content.

Direction-Level Data Instances. The complete dataset contains six independent direction-level tasks:

$$\begin{aligned} &\{\text{Hi} \rightarrow \text{Bhi}, \text{Bhi} \rightarrow \text{Hi}\}, \\ &\{\text{Hi} \rightarrow \text{Gon}, \text{Gon} \rightarrow \text{Hi}\}, \\ &\{\text{Hi} \rightarrow \text{Mun}, \text{Mun} \rightarrow \text{Hi}\}. \end{aligned} \tag{18}$$

Here, Bhi, Gon, and Mun denote Bhili, Gondi, and Mundari. RECAP treats each of these six tasks independently: each has its own stored four-model candidate rows, calibration statistics, preference dataset, existing mT5 SFT checkpoint used as the frozen reference policy, and final evaluation. This design permits a direct test of whether the method works for both Hindi-to-low-resource and low-resource-to-Hindi translation.

6.4 Observable Translation Reward

The phrase *multi-aspect* has a deliberately narrow meaning in this paper. For every candidate, RECAP uses three reference-aware quality metrics—BLEU, ChrF++, and COMET—plus directly observable repetition and length penalties. COMET is a fixed, reference-aware learned metric; it is not treated as human ground truth, a target-language fluency model, or a detector of hallucination. Its checkpoint, input format, score direction, and coverage for all six directions must be fixed before mining preferences. Low-resource evaluator limitations therefore remain a reporting and ablation concern [2, 4].

6.4.1 Conservative Automatic Reward Used in This Work

For candidate y of source x in direction d , let $B_d(y, y^*)$, $H_d(y, y^*)$, and $C_d(x, y, y^*)$ denote sentence-level BLEU, ChrF++, and COMET, respectively. BLEU and ChrF++ compare the candidate with the reference; COMET additionally uses the source. Let $P_{\text{rep}}(y)$ and $P_{\text{len}}(y, y^*)$ denote repetition and length mismatch. Every raw quantity is standardized using statistics estimated from training candidates for the same direction:

$$\tilde{s}_d(y) = \frac{s_d(y) - \mu_{s,d}}{\sigma_{s,d} + \epsilon}. \tag{19}$$

Here s can be any one of B , H , C , P_{rep} , or P_{len} ; $\mu_{s,d}$ and $\sigma_{s,d}$ are fitted only on training candidates for direction d , and $\epsilon > 0$ prevents division by zero. The standardized quality component gives the three metrics equal weight:

$$L_{\text{quality},d}(x, y, y^*) = \frac{\tilde{B}_d(y, y^*) + \tilde{H}_d(y, y^*) + \tilde{C}_d(x, y, y^*)}{3}. \tag{20}$$

The final reward uses three non-negative hyperparameters: w_{quality} , w_{rep} , and w_{len} . They are selected using validation data and then fixed before DPO, GRPO, or PPO training:

$$\boxed{R_d^{\text{RECAP}}(x, y, y^*) = w_{\text{quality}}L_{\text{quality},d}(x, y, y^*) - w_{\text{rep}}\tilde{P}_{\text{rep},d}(y) - w_{\text{len}}\tilde{P}_{\text{len},d}(y, y^*)}, \tag{21}$$

where

$$w_{\text{quality}}, w_{\text{rep}}, w_{\text{len}} \geq 0, \quad w_{\text{quality}} + w_{\text{rep}} + w_{\text{len}} = 1. \quad (22)$$

For example, one *illustrative* setting is $(w_{\text{quality}}, w_{\text{rep}}, w_{\text{len}}) = (0.60, 0.25, 0.15)$, and $0.60 + 0.25 + 0.15 = 1$. The weights are reward-design hyperparameters, not learned from the pseudo-preferences that they create. Their selected values and validation procedure must be reported, and they must never be tuned on the held-out test set.

The repetition penalty can be computed from repeated character n -grams (or word n -grams when dependable tokenization is available):

$$P_{\text{rep}}(y) = \frac{N_{\text{repeated}}(y)}{\max\{N_{\text{total}}(y), 1\}}. \quad (23)$$

Here, $N_{\text{total}}(y)$ is the total number of selected n -grams in y , and $N_{\text{repeated}}(y)$ is the number that occur more than once beyond their first occurrence. The denominator is clipped to at least 1 so that an empty or very short output never causes division by zero. A larger value means more repeated material. This detects an observable failure pattern; it does not require knowing the language.

The length penalty is

$$P_{\text{len}}(y, y^*) = \left| \frac{|y|}{\max\{|y^*|, 1\}} - 1 \right|, \quad (24)$$

where $|y|$ and $|y^*|$ are the character counts or token counts of the candidate and reference, measured consistently within a direction. If the two lengths are equal, the ratio is 1 and the penalty is 0. For example, if a candidate has 150 characters and its 100-character reference has no zero-length issue, then $P_{\text{len}} = |150/100 - 1| = 0.5$. This is a warning signal for severe truncation or verbosity, not proof that a translation is wrong.

Worked Example: Measuring Every Automatic Component. For one illustrative candidate, suppose the direction-standardized values are $\tilde{B} = 0.60$, $\tilde{H} = 1.00$, $\tilde{C} = 0.80$, $\tilde{P}_{\text{rep}} = 0.50$, and $\tilde{P}_{\text{len}} = -0.20$. If validation selected the illustrative hyperparameter setting $(0.60, 0.25, 0.15)$ above, then

$$\begin{aligned} L_{\text{quality}} &= \frac{0.60 + 1.00 + 0.80}{3} = 0.80, \\ R^{\text{RECAP}} &= 0.60(0.80) - 0.25(0.50) - 0.15(-0.20) = 0.385. \end{aligned} \quad (25)$$

The candidate is above average on all three quality metrics, has above-average repetition, and has slightly below-average length mismatch. Different validation-selected weights would give a different numerical reward. In all cases, the reward is a relative ranking signal within this direction, not a percentage and not a human quality judgment.

6.4.2 What Cannot Be Detected Reliably from the Available Data Alone

COMET limitations. COMET contributes a learned, reference-aware quality signal, but its score is not a human preference label. It can be affected by language coverage, domain mismatch, and evaluator bias. RECAP therefore fixes one COMET configuration before training, reports it separately at test time, and does not interpret it as proof of semantic adequacy.

Semantic adequacy. A low reference-fidelity score can indicate that a candidate differs substantially from y^* , but it cannot tell whether the difference is a valid paraphrase or a meaning error. Therefore, L_{ref} is called a *reference-fidelity proxy*, not a semantic-adequacy detector.

Fluency. Judging whether a Bhili, Gondi, or Mundari sentence is grammatical and natural requires a validated target-language language model or qualified speakers. Neither is available under the stated assumptions. RECAP therefore does not include an automatic fluency score in its default reward. Repetition and extreme-length checks catch only a small set of visible form errors; they are not fluency measures.

Hallucination. Calling an addition “unsupported by the source” requires semantic comparison of the source and candidate across languages. Reference overlap alone cannot make that judgment: a valid paraphrase may have low overlap, and a candidate can share many reference words while changing its meaning. RECAP consequently does not use a claimed hallucination detector in the default reward. It records repetition and reference divergence separately, and reserves the term *hallucination* for later human or independently validated analysis.

Target-language correctness. A language-ID score is also excluded unless a direction-specific classifier is independently shown to distinguish Bhili, Gondi, Mundari, Hindi, and code-mixed text. Shared scripts make script detection insufficient: Devanagari characters alone do not prove that an output is Bhili, Gondi, or Hindi.

Thus, the automatic DPO labels in this zero-human-label setting mean only: “under fixed BLEU, ChrF++, COMET, repetition, and length criteria, this candidate receives a higher reward than the other candidate.” They do *not* mean “a human verified that this candidate is more faithful, fluent, or free of hallucination.”

6.4.3 Optional Components Require Prior Validation

Semantic similarity, fluency, hallucination, and language-correctness components may be added only if a separately collected, direction-specific validation set establishes that the proposed detector agrees sufficiently with qualified human judgments. Such validation must be performed before using the component to construct DPO pairs, and its coverage, errors, and correlation with human ratings must be reported. Until then, RECAP uses the conservative automatic reward above and reports its held-out improvement only in terms of reference-based translation metrics and observable degeneration checks.

The reward weights are selected as validation hyperparameters before training and never tuned on the held-out test set. A sensitivity analysis should report whether nearby convex settings alter retained pairs, preference consistency, or final held-out results.

6.5 Direction Calibration, Disagreement, and Diagnostics

Raw scores are not directly comparable across language directions. Calibration is therefore performed separately for the five raw quantities $\mathcal{S} = \{B, H, C, P_{\text{rep}}, P_{\text{len}}\}$: BLEU, ChrF++, COMET, repetition, and length mismatch. For every $s \in \mathcal{S}$, we estimate its mean and standard deviation from training candidates in direction d :

Illustrative example: equal BLEU does not imply equal meaning. Consider the Hindi source sentence “बैठक कल नहीं, परसों होगी।” (“The meeting will be the day after tomorrow, not tomorrow.”). The numbers below are hypothetical and are used only to explain calibration.

To make the semantic error explicit, consider the following English glosses of two hypothetical candidate translations:

- **Correct meaning:** “The meeting will be the day after tomorrow, not tomorrow.”
- **Wrong candidate:** “The meeting will be tomorrow, not the day after tomorrow.” The words *tomorrow* and *the day after tomorrow* are reversed, so the meeting date is wrong.

In Hindi $\rightarrow$ Bhili, this wrong candidate can still share many surface forms with the reference and receive a raw BLEU value of 30. In the reverse Bhili $\rightarrow$ Hindi direction, a correct paraphrase such as “Not tomorrow—the meeting is the day after tomorrow” can express the same meaning with different wording and also receive a raw BLEU value of 30.

Therefore, a raw BLEU of 30 is not a universal statement about translation quality. The Hindi $\rightarrow$ Bhili score should be interpreted relative to the normal BLEU distribution for Hindi $\rightarrow$ Bhili, and the Bhili $\rightarrow$ Hindi score relative to the distribution for Bhili $\rightarrow$ Hindi. Calibration makes these direction-specific interpretations possible, but it does *not* resolve the semantic ambiguity illustrated above. Under the stated assumptions, RECAP treats this as a limitation of reference-based scoring rather than claiming to detect the changed date automatically.

$$\mu_{s,d} = \mathbb{E}_{\text{train},d}[s], \quad \sigma_{s,d} = \text{Std}_{\text{train},d}[s]. \quad (26)$$

We then standardize each value and form the equal-weight quality component:

$$\tilde{s}_d = \frac{s - \mu_{s,d}}{\sigma_{s,d} + \epsilon}, \quad L_{\text{quality},d}(x, y, y^*) = \frac{\tilde{B}_d(y, y^*) + \tilde{H}_d(y, y^*) + \tilde{C}_d(x, y, y^*)}{3}. \quad (27)$$

The complete reward is $w_{\text{quality}}L_{\text{quality},d} - w_{\text{rep}}\tilde{P}_{\text{rep},d} - w_{\text{len}}\tilde{P}_{\text{len},d}$. Thus, a large positive standardized quality value helps a candidate, whereas large positive standardized penalties lower its reward.

The automatic reward directly models only observable repetition and length mismatch in addition to reference fidelity. For example,

$$P_{\text{rep}} = \frac{N_{\text{repeated ngrams}}}{\max\{N_{\text{total ngrams}}, 1\}}, \quad P_{\text{len}} = \left| \frac{|y|}{\max\{|y^*|, 1\}} - 1 \right|. \quad (28)$$

COMET is standardized exactly like BLEU and ChrF++ but remains separately reportable at evaluation time; standardization is used only for the training-time reward.

6.5.1 Calibration Procedure

Calibration is estimated only from training or validation data for the same direction. For each reward component, we first collect scores for a representative set of candidate translations. We then estimate $\mu_{d,k}$ and $\sigma_{d,k}$, add a small constant ϵ for numerical stability, and convert every raw component to a standardized value. This prevents a component with a large numerical range from dominating another component only because of its scale.

In practice, calibration should be checked for outliers. A few very poor candidates can make a standard deviation large and reduce the contrast among normal candidates. If this happens, robust alternatives such as median–interquartile-range normalization or score clipping can be evaluated. The important requirement is that the chosen calibration method is fit separately for each direction and is not adjusted on the final test set.

The reward should be decomposed and saved for every candidate:

$$\mathbf{r}(x, y, y^*) = (B, H, C, P_{\text{rep}}, P_{\text{len}}, R^{\text{RECAP}}). \quad (29)$$

Storing this vector makes it possible to explain why one candidate was preferred. For example, a candidate with strong quality metrics can still be rejected if it has substantially more repetition or a large length mismatch than another candidate.

6.5.2 Quantifying Candidate Disagreement

For one source x_i , let the four heterogeneous candidates and their RECAP rewards be

$$\mathcal{C}_i = \{(m_k, y_{i,k}, R_{i,k})\}_{k=1}^4, \quad R_{i,k} = R_d^{\text{RECAP}}(x_i, y_{i,k}, y_i^*). \quad (30)$$

This one notation is sufficient for both disagreement analysis and pair construction: m_k identifies NLLB, mT5, Qwen, or Llama; $y_{i,k}$ is that model’s translation; and $R_{i,k}$ is its final direction-calibrated reward. We summarize the group by four complementary quantities:

$$\begin{aligned} \bar{R}_i &= \frac{1}{4} \sum_{k=1}^4 R_{i,k}, \\ D_{\text{range}}(x_i) &= \max_k R_{i,k} - \min_k R_{i,k}, \\ D_{\text{sd}}(x_i) &= \sqrt{\frac{1}{4} \sum_{k=1}^4 (R_{i,k} - \bar{R}_i)^2}, \\ M_{\text{top}}(x_i) &= R_{i,(1)} - R_{i,(2)}, \end{aligned} \quad (31)$$

where $R_{i,(1)} \geq R_{i,(2)} \geq \dots \geq R_{i,(4)}$ are the same four rewards sorted from largest to smallest. The range measures the largest available better–worse gap, the standard deviation measures overall spread, and the top-two margin measures whether the nominal winner is clearly separated from the runner-up.

For a complete worked calculation, suppose the four candidates receive the following dummy rewards:

Generator	$R_{i,k}$	$R_{i,k} - \bar{R}_i$	$(R_{i,k} - \bar{R}_i)^2$
NLLB	0.18	−0.1075	0.01155625
mT5	0.46	0.1725	0.02975625
Qwen	−0.35	−0.6375	0.40640625
Llama	0.86	0.5725	0.32775625
Sum	1.15		0.775475

First calculate the mean reward:

$$\bar{R}_i = \frac{0.18 + 0.46 + (-0.35) + 0.86}{4} = \frac{1.15}{4} = 0.2875. \quad (32)$$

Using this mean, the table shows every deviation and squared deviation needed for the population standard deviation. The remaining disagreement quantities are

$$\begin{aligned}
D_{\text{range}}(x_i) &= \max(0.18, 0.46, -0.35, 0.86) - \min(0.18, 0.46, -0.35, 0.86) \\
&= 0.86 - (-0.35) = 1.21, \\
D_{\text{sd}}(x_i) &= \sqrt{\frac{0.01155625 + 0.02975625 + 0.40640625 + 0.32775625}{4}} \\
&= \sqrt{\frac{0.775475}{4}} = \sqrt{0.19386875} \approx 0.4403, \\
M_{\text{top}}(x_i) &= R_{i,(1)} - R_{i,(2)} = 0.86 - 0.46 = 0.40.
\end{aligned} \tag{33}$$

Thus, this source has a range of 1.21, a group spread of approximately 0.44, and a clear Llama-mT5 top-two margin of 0.40. RECAP uses disagreement only as a diagnostic or optional sampling priority: it can prioritize sources with large D_{range} or D_{sd} , but it does not itself create a preference label. Labels are created later from every pairwise margin $R_{i,a} - R_{i,b}$ that exceeds the confidence threshold δ . This distinction prevents a large best-worst spread from being confused with a requirement to train only on the best-worst pair.

6.5.3 Observable Diagnostics

The automatic pipeline records three diagnostic categories:

Reference-aware quality

Sentence-level BLEU, ChrF++, and fixed-configuration COMET, each computed against the available reference (and source where required by COMET).

Repetition Repeated character or word n -grams.

Length mismatch

A large candidate-reference length difference.

These diagnostics are sufficient to explain the automatic pseudo-label, but they are not a complete taxonomy of translation errors.

If qualified human evaluators become available later, they may perform a separate qualitative analysis of errors that the automatic reward cannot identify. That analysis is external validation; it is not used to define the default DPO labels in this work.

6.6 Preference Mining and Pair Filtering

For each source x_i , RECAP evaluates all unordered pairs among its $K = 4$ candidates. For pair (a, b) with $a < b$, the signed reward difference is

$$\Delta_{i,a,b} = R_{i,a} - R_{i,b}. \tag{34}$$

If $|\Delta_{i,a,b}| > \delta$, RECAP keeps the pair and orients it according to the reward: if $\Delta_{i,a,b} > 0$, then $(y_{i,a}, y_{i,b})$ becomes (y^+, y^-) ; otherwise, $(y_{i,b}, y_{i,a})$ becomes (y^+, y^-) . Exact or normalized-text duplicates are discarded. Thus, the valid-pair set for source i is

$$\mathcal{P}_i = \{(x_i, y^+, y^-) : 1 \leq a < b \leq 4, |\Delta_{i,a,b}| > \delta\}. \tag{35}$$

The value δ is a confidence margin, not a quality score: it prevents a small reward difference from being treated as a strong pseudo-preference. A source can contribute between zero and six

valid DPO pairs. Disagreement may prioritize sources for sampling, but every retained pair must independently pass this pairwise margin test.

6.6.1 All-Pairs Margin Filtering and Balanced Sampling

With four candidate generators, each source has

$$\binom{4}{2} = 6 \quad (36)$$

possible unordered model pairs. All six are considered before filtering. This is preferable to always retaining only the best–worst pair because a repeated strong-versus-weak pairing can make DPO imitate one generator rather than learn from heterogeneous candidate strengths.

After margin filtering, RECAP balances *sampling of valid pairs*, not preference directions. Let $g = \{m_a, m_b\}$ denote one of the six unordered generator-pair types and let $\mathcal{P}_{d,g}$ be its valid pairs in direction d . The final DPO training set is sampled with a per-type cap q_d :

$$\mathcal{P}_d^{\text{DPO}} = \bigcup_g \text{Sample}(\mathcal{P}_{d,g}, \min\{|\mathcal{P}_{d,g}|, q_d\}). \quad (37)$$

This procedure never reverses a pair merely to equalize counts. It preserves the reward-derived $y^+ \succ y^-$ direction while preventing an abundant pair type such as mT5–Qwen from dominating all DPO updates. The cap is chosen on validation data and reported with the final dataset statistics.

For auditability, let n_m^+ and n_m^- be the numbers of retained pairs in which model m generated the preferred and rejected output, respectively. With N_{pair} retained pairs, RECAP reports model participation as

$$C_m^+ = \frac{n_m^+}{N_{\text{pair}}}, \quad C_m^- = \frac{n_m^-}{N_{\text{pair}}}. \quad (38)$$

The goal is not to force every model to appear exactly 25% of the time. It is to reveal and control pathological collapse, such as one model being preferred in nearly every pair and another being rejected in nearly every pair. If $N_{\text{candidate comparisons}}$ is the number of valid unordered comparisons examined and $N_{\text{retained pairs}}$ is the number that pass the margin, deduplication, and sampling rules, then

$$\text{RetentionRate} = \frac{N_{\text{retained pairs}}}{N_{\text{candidate comparisons}}}. \quad (39)$$

6.6.2 Why Sentence-Level Scores Are Required for Preference Mining

Yes: RECAP requires a reward for *each candidate translation of each source sentence*. For source x_i and candidate model j , the algorithm needs

$$R_{i,j} = R_d^{\text{RECAP}}(x_i, y_{i,j}, y_i^*), \quad (40)$$

so that it can compare $R_{i,j}$ with the rewards of the other candidates for the same x_i . A single corpus-level BLEU or ChrF++ value has one number for an entire system over many sources. It cannot decide whether NLLB or mT5 produced the better candidate for one particular source, so it cannot create the pair (x_i, y_i^+, y_i^-) required by DPO.

Therefore, BLEU, ChrF++, COMET, repetition, and length mismatch are all calculated per candidate sentence during preference mining. Held-out corpus-level BLEU, ChrF++, and COMET

are reported separately for final evaluation because they summarize the quality of a complete system over the test set. Sentence-level BLEU can be sparse for short translations, so RECAP must use one fixed smoothing method for sentence BLEU, record that method, and apply it identically to every candidate and direction. ChrF++, COMET, and the repetition and length components provide additional sentence-level signal, while margin filtering removes close, unstable rankings.

Preference pairs should also be deduplicated. If two different sources produce identical outputs, or if the preferred and rejected outputs are identical after normalization, the pair does not provide useful preference information. The final dataset should retain the source, reference, both candidates, generator identities, all reward components, margin, confidence, and split assignment.

7 Training Algorithms

7.1 Direct Preference Optimization

RECAP first uses offline DPO. For a preferred translation y_w and rejected translation y_l , the DPO loss is

$$\mathcal{L}_{\text{DPO}} = -\log \sigma \left[\beta \left(\log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]. \quad (41)$$

7.1.1 What the Trainable and Reference Policies Mean

In this paper, a *policy* is simply a translation model viewed as a conditional probability distribution. Given a source sentence x , it assigns a probability to every possible target translation y .

$\pi_{\theta}(y | x)$ The *current trainable policy*. It is the translation model whose parameters are denoted by θ . The symbol θ means all of the model’s learned weights. During DPO, gradient descent changes θ , so π_{θ} gradually assigns more probability to preferred translations and less probability to rejected translations.

$\pi_{\text{ref}}(y | x)$ The *reference policy*. It is a frozen copy of the direction-specific SFT model at the start of DPO. Its weights never change during DPO. It provides a stable baseline for measuring how much the trainable policy has changed its preference for a translation.

For a direction d , the initialization is therefore

$$\pi_{\theta,d} \leftarrow \pi_{\text{SFT},d}, \quad \pi_{\text{ref},d} = \pi_{\text{SFT},d} \text{ and is frozen.} \quad (42)$$

The first expression means that the trainable DPO model starts from the SFT checkpoint. The second means that a separate, unchanged copy of that same SFT checkpoint is retained as $\pi_{\text{ref},d}$. Because RECAP treats directions independently, Bhili $\rightarrow$ Hindi has its own $\pi_{\theta,d}$ and $\pi_{\text{ref},d}$; they are not shared with, for example, Gondi $\rightarrow$ Hindi.

Probability of a complete translation. Let $y = (y_1, y_2, \dots, y_T)$ be a translation containing T generated tokens. The model generates tokens one at a time. Its probability for the complete translation is

$$\pi_{\theta}(y | x) = \prod_{t=1}^T \pi_{\theta}(y_t | x, y_{<t}), \quad (43)$$

where y_t is token t and $y_{<t} = (y_1, \dots, y_{t-1})$ is the partial translation already generated before token t . The product symbol $\prod$ means “multiply the probability of every generated token.” In implementation, the model uses log-probabilities because products of many small probabilities become extremely small:

$$\log \pi_\theta(y \mid x) = \sum_{t=1}^T \log \pi_\theta(y_t \mid x, y_{<t}). \quad (44)$$

Why DPO divides by the reference policy. For any candidate y , define its change relative to SFT as

$$g_\theta(x, y) = \log \pi_\theta(y \mid x) - \log \pi_{\text{ref}}(y \mid x) = \log \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}. \quad (45)$$

If $g_\theta(x, y) > 0$, the DPO policy gives y more probability than the frozen SFT model did. If $g_\theta(x, y) < 0$, it gives y less probability. The DPO loss pushes the difference

$$g_\theta(x, y_w) - g_\theta(x, y_l) \quad (46)$$

to be positive: relative to SFT, the trainable policy should increase probability for the chosen translation y_w more than for the rejected translation y_l .

Small numerical example. Suppose a source x has a chosen translation y_w and a rejected translation y_l . The following values are dummy probabilities used only to explain the calculation:

$$\pi_\theta(y_w \mid x) = 0.004, \quad \pi_{\text{ref}}(y_w \mid x) = 0.002, \quad \pi_\theta(y_l \mid x) = 0.001, \quad \pi_{\text{ref}}(y_l \mid x) = 0.002. \quad (47)$$

For the chosen translation, the trainable policy is twice as likely as the reference policy:

$$g_\theta(x, y_w) = \log \frac{0.004}{0.002} = \log 2 \approx 0.693. \quad (48)$$

For the rejected translation, the trainable policy is only half as likely as the reference policy:

$$g_\theta(x, y_l) = \log \frac{0.001}{0.002} = \log 0.5 \approx -0.693. \quad (49)$$

The DPO preference gap is therefore

$$g_\theta(x, y_w) - g_\theta(x, y_l) = 0.693 - (-0.693) = 1.386. \quad (50)$$

This positive value means that the current policy has moved in the desired direction: compared with its SFT baseline, it favors y_w over y_l . The inverse-temperature β controls how strongly this gap affects the loss, and $\sigma(\cdot)$ converts the scaled gap into a value between 0 and 1. Importantly, neither π_θ nor π_{ref} is a translation-quality metric; the automatic reward constructs the pseudo-preference, while DPO uses the two policies to learn from that preference.

7.1.2 Selecting One Common SFT Backbone

For the main controlled experiment, RECAP uses **mT5** as the common SFT and DPO backbone across all six directions. The four candidate columns are already available SFT outputs; mT5 is selected only as the existing checkpoint from which DPO, GRPO, and PPO are initialized. This choice uses the validation columns of Tables 3–5; it is not chosen from test results or a few individual example rows.

For every candidate model $m \in \mathcal{M}_{\text{trainable}}$ and direction $d \in \mathcal{D}$, let $\text{BLEU}_{m,d}^{\text{val}}$ and $\text{ChrF} + +_{m,d}^{\text{val}}$ denote validation scores on the 0–100 scale. Define the model’s macro-average validation score as

$$S_m = \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} \left[\frac{1}{2} \frac{\text{BLEU}_{m,d}^{\text{val}}}{100} + \frac{1}{2} \frac{\text{ChrF} + +_{m,d}^{\text{val}}}{100} \right]. \quad (51)$$

In this equation, $|\mathcal{D}| = 6$ is the number of translation directions. BLEU and ChrF++ are first rescaled to $[0, 1]$ and receive equal weight; the outer average gives every direction equal importance, so a larger corpus does not dominate the decision. This is the appropriate selection rule because Tables 3–5 provide these two validation metrics for every candidate model and direction. COMET remains part of the RECAP reward and final evaluation, but it is not retroactively used to select a backbone from tables that do not report COMET.

The selected common backbone is

$$m^* = \arg \max_{m \in \mathcal{M}_{\text{trainable}}} S_m. \quad (52)$$

The notation $\arg \max$ means “select the model with the largest value of S_m .” Applying this rule to Tables 3–5 yields $m^* = \text{mT5}$: its macro-average validation BLEU is 41.97 and its macro-average validation ChrF++ is 61.23, both higher than the corresponding averages of the other candidate models. The reason for using mT5 is therefore its strongest average validation performance across the full six-direction study, not a claim that it wins every individual direction.

Choosing mT5 as the common backbone does *not* mean training one shared checkpoint on all directions. RECAP remains direction-isolated. For every direction d , we load the existing direction-specific mT5 SFT checkpoint:

$$\pi_{\text{SFT},d} = \pi_{\text{mT5,SFT},d}^{\text{existing}}, \quad \pi_{\theta,d} \leftarrow \pi_{\text{SFT},d}, \quad \pi_{\text{ref},d} = \text{Freeze}(\pi_{\text{SFT},d}). \quad (53)$$

Here, $\pi_{\text{mT5,SFT},d}^{\text{existing}}$ denotes the pre-existing mT5 SFT checkpoint for direction d , whose stored output is the y^{mT5} candidate column. The second expression initializes the trainable DPO policy from that checkpoint. The third creates its frozen reference copy. Thus, one selected backbone family yields six independent DPO runs, not one multilingual DPO policy. NLLB, mT5, Qwen, and Llama all remain in the fixed heterogeneous candidate pool used to construct preference pairs, including the selected mT5 family itself. The compact candidate-validation summary is reported in Table 6.

RECAP can weight this loss by confidence:

$$\mathcal{L}_{\text{RECAP-DPO}} = -c_i \log \sigma [\beta(\Delta \log \pi_{\theta} - \Delta \log \pi_{\text{ref}})]. \quad (54)$$

The reference model is the SFT model for the same direction: $\pi_{\text{ref},d} = \pi_{\text{SFT},d}$. Thus, each direction follows $\text{SFT}_d \rightarrow \text{RECAP-DPO}_d$. DPO is the primary optimizer in RECAP because it directly tests whether reliable pseudo-preferences can improve translation over the SFT starting point.

7.1.3 DPO Training Details

For a fair experiment, DPO should start from the same SFT checkpoint used as the reference model. The preference dataset must be split by source sentence before training so that a source appearing in the validation or test set does not also appear in the DPO training pairs. Training should monitor the DPO loss, mean preference margin, KL divergence from the reference, validation translation metrics, and the rate of degenerate outputs.

The inverse-temperature parameter β determines how strongly the model follows the preference labels. Very large values can overfit noisy pseudo-preferences; very small values may produce little change from SFT. Therefore, β , learning rate, batch size, number of epochs, maximum sequence length, and confidence weighting should be selected using direction-specific validation data. The model with the best validation criterion, not the final training-step checkpoint by default, should be used for test evaluation.

7.2 Online Refinement with GRPO

GRPO is an online refinement stage that begins after RECAP-DPO. For direction d , initialize the trainable policy and its frozen KL reference from the selected DPO checkpoint:

$$\pi_{\theta,d}^{(0)} \leftarrow \pi_{\text{DPO},d}, \quad \pi_d^{\text{GRPO-ref}} = \text{Freeze}(\pi_{\text{DPO},d}). \quad (55)$$

For each selected source x_i , only the current policy stochastically generates a fresh group of G translations:

$$y_{i,1}, \dots, y_{i,G} \sim \pi_{\theta,d}(\cdot \mid x_i). \quad (56)$$

We evaluate $G \in \{2, 3\}$, with sampling temperature, nucleus probability, maximum length, and repetition control fixed and logged. Each generated translation receives the same RECAP reward against the training reference y_i^* , and GRPO uses the relative reward within its own group. We compute

$$\mu_R = \frac{1}{G} \sum_{i=1}^G R_i, \quad \sigma_R = \sqrt{\frac{1}{G} \sum_{i=1}^G (R_i - \mu_R)^2}, \quad (57)$$

and the relative advantage

$$A_i = \frac{R_i - \mu_R}{\sigma_R + \epsilon}. \quad (58)$$

GRPO encourages candidates with positive relative advantage and discourages those with negative relative advantage. After each update, the newly updated policy generates the next fresh group. Thus, the sequence is RECAP-DPO checkpoint, stochastic online generation, RECAP reward evaluation, relative-advantage calculation, and GRPO update. The fixed offline DPO pairs are not reused as GRPO candidate groups.

7.2.1 Online RL Safeguards

Online optimization can exploit weak points in an automatic reward. To reduce this risk, each GRPO update should be monitored with BLEU, ChrF++, COMET, repetition, length mismatch, and total reward. An increase in total reward accompanied by worse repetition or length mismatch is a warning that the model may be reward hacking. Generation length, decoding temperature,

group size G , update frequency, and KL coefficient should be logged and kept comparable across experiments.

Early stopping can be based on validation BLEU, ChrF++, COMET, repetition, and length-mismatch checks. If qualified inspection becomes available, it should be reported separately and should not be presented as part of the automatic reward. If a policy becomes unstable, the experiment should report the failure mode rather than selecting a favorable checkpoint after the fact.

7.3 PPO Baseline and KL Regularization

PPO is a classical on-policy reinforcement-learning baseline. It is included to answer a controlled question: *if the same automatic RECAP reward is optimized directly from SFT, does PPO improve translation as much as preference-based RECAP-DPO or RECAP-DPO+GRPO?* Thus, PPO isolates the effect of the optimizer. Unlike DPO, PPO does not receive offline triples (x, y^+, y^-) and does not use the four heterogeneous candidates after initialization. Its training path is

$$\boxed{\text{mT5-SFT}_d \longrightarrow \text{PPO}_d \longrightarrow \pi_{\text{PPO},d}} \quad (59)$$

7.3.1 PPO Inputs, Outputs, and Automatic Reward

For one direction d , a PPO training batch contains source–reference pairs (x_i, y_i^*) from $\mathcal{T}_d^{\text{train}}$. The trainable translation policy starts from the direction-specific mT5 SFT checkpoint,

$$\pi_{\theta,d}^{(0)} \leftarrow \pi_{\text{SFT},d}, \quad \pi_{\text{ref},d} = \text{Freeze}(\pi_{\text{SFT},d}). \quad (60)$$

For each source x_i , the current policy samples one translation $\hat{y}_i \sim \pi_{\theta,d}(\cdot | x_i)$ and stores its token-level actions, log-probabilities, and value estimates. The output of a PPO rollout is therefore a set of generated translations with the information needed to update the policy; the output after all updates is the checkpoint $\pi_{\text{PPO},d}$.

There is **no separately trained reward model** in this PPO baseline. The word “reward” refers to the fixed automatic RECAP reward function already used for DPO preference mining. After $\hat{y}_i$ has been generated, the held training reference y_i^* is used only to score it:

$$\begin{aligned} R_i^{\text{PPO}} &= R_d^{\text{RECAP}}(x_i, \hat{y}_i, y_i^*) \\ &= w_{\text{quality}} \left(\frac{\tilde{B}_d(\hat{y}_i, y_i^*) + \tilde{H}_d(\hat{y}_i, y_i^*) + \tilde{C}_d(x_i, \hat{y}_i, y_i^*)}{3} \right) \\ &\quad - w_{\text{rep}} \tilde{P}_{\text{rep},d}(\hat{y}_i) - w_{\text{len}} \tilde{P}_{\text{len},d}(\hat{y}_i, y_i^*). \end{aligned} \quad (61)$$

In simple terms, Equation (62) gives PPO one score for the translation it has just produced. First, it checks how closely the translation matches the gold reference using BLEU, ChrF++, and COMET, and averages these three quality signals. It then subtracts points if the translation repeats material or is much shorter or longer than the reference. A translation with stronger reference-based quality and fewer observable problems receives a larger reward, so PPO is encouraged to make translations like that more probable in later updates. The three w values are validation-selected reward hyperparameters: one weights quality and the other two weight the penalties.

This equation scores PPO’s newly generated translation. The symbols mean:

i, d i identifies one rollout example and d identifies one translation direction. Standardization is performed separately for each d .

$x_i, \hat{y}_i, y_i^*$ x_i is the source, $\hat{y}_i$ is PPO’s sampled translation, and y_i^* is its gold reference. The reference is used only after generation to calculate reward.

$R_i^{\text{PPO}} = R_d^{\text{RECAP}}(\cdot)$ The scalar reward for $\hat{y}_i$. PPO reuses the fixed RECAP reward; it does not train a separate reward model.

B, H, C Sentence-level BLEU and ChrF++ compare $\hat{y}_i$ with y_i^* ; COMET uses $x_i, \hat{y}_i$, and y_i^* . Their tilded forms are direction-standardized scores.

$\sim$ A tilde denotes z-score standardization:

$$\tilde{s}_d = \frac{s_d - \mu_{s,d}}{\sigma_{s,d} + \epsilon}. \quad (62)$$

Here $\mu_{s,d}$ and $\sigma_{s,d}$ are the direction-specific training-candidate mean and standard deviation, and $\epsilon > 0$ prevents division by zero.

Quality and penalties

$(\tilde{B}_d + \tilde{H}_d + \tilde{C}_d)/3$ gives the three quality metrics equal weight. $\tilde{P}_{\text{rep},d}$ and $\tilde{P}_{\text{len},d}$ measure repetition and length mismatch; larger values are worse. The validation-selected hyperparameters w_{quality} , w_{rep} , and w_{len} weight quality and subtract the two penalties.

Here BLEU, ChrF++, and COMET form the quality component, while repetition and length mismatch are penalties. Direction-specific standardization statistics are estimated before PPO begins. The quality, repetition, and length weights are fixed in advance and remain unchanged during PPO. The reference is *not* supplied to the model as an input when it produces $\hat{y}_i$; it is used afterwards only to compute the scalar training reward. This makes PPO a reference-based offline RL experiment, not RL from human feedback.

PPO also uses a learned *value head* $V_\phi(s_t)$, which predicts the expected future reward from the partial translation state $s_t = (x_i, \hat{y}_{i,<t})$. The value head is not a reward model: it does not decide whether a translation is good. It reduces the variance of the policy-gradient estimate so that the fixed RECAP reward can be used more stably.

7.3.2 One PPO Training Update

Let $a_{i,t} = \hat{y}_{i,t}$ be token t in the sampled translation and let T_i be its final length. At collection time, freeze the current policy as $\pi_{\theta_{\text{old},d}}$ and generate a rollout batch. The translation-level RECAP reward is assigned at the final token:

$$r_{i,t}^{\text{env}} = \begin{cases} 0, & t < T_i, \\ R_i^{\text{PPO}}, & t = T_i. \end{cases} \quad (63)$$

In simple terms, PPO generates a translation one token at a time, but RECAP can judge its quality only after the entire translation is complete. Here t is the current token position, T_i is the final token position of translation i (the point at which the model produces its end-of-sentence token), and $r_{i,t}^{\text{env}}$ is the immediate environment reward assigned at token position t . For every earlier token, where $t < T_i$, the immediate reward is set to 0: this does *not* mean that those tokens are unimportant, only that the full translation has not yet been scored. At the final token, where

$t = T_i$, PPO receives the complete translation-level score R_i^{PPO} from Equation (62). The later advantage calculation propagates this final reward back to the earlier token choices, so tokens that helped produce a high-reward translation are reinforced and tokens associated with a low-reward translation are discouraged.

To keep PPO close to SFT, we subtract a token-level KL penalty relative to the frozen SFT reference policy:

$$\tilde{r}_{i,t} = r_{i,t}^{\text{env}} - \beta_{\text{KL}} [\log \pi_{\theta_{\text{old},d}}(a_{i,t} \mid s_{i,t}) - \log \pi_{\text{ref},d}(a_{i,t} \mid s_{i,t})]. \quad (64)$$

In simple terms, this equation takes the translation reward from the previous equation and subtracts a “stay close to SFT” cost. The term β_{KL} controls how strongly this cost matters. At token position t , $a_{i,t}$ is the token PPO chose, $s_{i,t}$ is the source plus all earlier generated tokens, $\pi_{\theta_{\text{old},d}}$ is the policy that generated the rollout, and $\pi_{\text{ref},d}$ is the frozen mT5 SFT policy. If the rollout policy assigns much more probability to a token than the frozen SFT policy does, the bracketed difference is positive and the penalty reduces the reward. Thus, $\tilde{r}_{i,t}$ is the reward actually used for PPO learning: it favors high-quality complete translations but discourages the policy from changing too far from its reliable SFT starting point.

The first term rewards the complete translation according to RECAP; the second discourages the updated policy from drifting too far from its SFT starting point. Given the value head, PPO computes temporal-difference residuals and generalized-advantage estimates:

$$\delta_{i,t} = \tilde{r}_{i,t} + \gamma V_{\phi_{\text{old}}}(s_{i,t+1}) - V_{\phi_{\text{old}}}(s_{i,t}), \quad A_{i,t} = \sum_{l=0}^{T_i-t} (\gamma\lambda)^l \delta_{i,t+l}. \quad (65)$$

In simple terms, PPO now asks two questions for each token choice. First, $\delta_{i,t}$ asks: “Was the outcome after this token better or worse than the value head expected?” The quantity $V_{\phi_{\text{old}}}(s_{i,t})$ is the old value head’s prediction before token t , while $V_{\phi_{\text{old}}}(s_{i,t+1})$ is its prediction after moving to the next partial translation state. The discount factor γ controls how much future reward counts. Second, $A_{i,t}$ combines this token’s prediction error with later prediction errors until the translation ends at T_i . The factor λ gradually downweights errors farther in the future. A positive advantage means the token sequence led to a better outcome than expected, so PPO increases its probability; a negative advantage means the outcome was worse than expected, so PPO decreases its probability.

For sentence-level translation rewards, $\gamma = 1$ is a natural default because the complete generated sentence receives one terminal reward; λ controls the bias–variance trade-off of the advantage estimate. A positive $A_{i,t}$ means that the sampled token sequence led to a better-than-expected outcome, whereas a negative value means it led to a worse-than-expected outcome.

At decoding step t , PPO compares the newly updated policy to the rollout policy⁷ through the probability ratio

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(a_{i,t} \mid s_{i,t})}{\pi_{\theta_{\text{old}}}(a_{i,t} \mid s_{i,t})}, \quad (66)$$

This $r_{i,t}(\theta)$ is a *probability ratio*, not a translation reward. It asks how much the updated policy has changed its probability for the particular token $a_{i,t}$ in the same partial translation state $s_{i,t}$. The numerator is the probability assigned by the policy currently being updated, while the denominator is the probability assigned by the frozen rollout policy that originally generated the

⁷The *rollout policy* is a frozen copy of PPO used to generate the current batch of translations before any update is made. PPO keeps it unchanged while learning from that batch, so it can measure exactly how much the new policy has changed.

training batch. If the ratio is 1, the token probability has not changed; if it is greater than 1, the updated policy makes that token more likely; and if it is less than 1, it makes that token less likely. For example, $r_{i,t}(\theta) = 1.10$ means a 10% increase relative to the rollout policy. PPO uses this ratio to improve preferred token choices while the later clipping rule prevents any one update from changing them too sharply.

and maximizes the clipped surrogate objective

$$\mathcal{L}_{\text{clip}} = \mathbb{E}_{i,t} [\min(r_{i,t}(\theta)A_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon)A_{i,t})]. \quad (67)$$

The clipping interval $[1 - \epsilon, 1 + \epsilon]$ prevents a single minibatch from changing a token probability too aggressively. In practice, PPO minimizes the following loss over several minibatch epochs from the same rollout batch:

$$\mathcal{L}_{\text{total}} = -\mathcal{L}_{\text{clip}} + c_V \mathcal{L}_{\text{value}} - c_H \mathcal{H}(\pi_\theta). \quad (68)$$

The value target and value loss are

$$\hat{G}_{i,t} = A_{i,t} + V_{\phi_{\text{old}}}(s_{i,t}), \quad (69)$$

$$\mathcal{L}_{\text{value}} = \mathbb{E}_{i,t} \left[\left(V_\phi(s_{i,t}) - \hat{G}_{i,t} \right)^2 \right]. \quad (70)$$

The value-loss coefficient c_V , optional entropy coefficient c_H , clipping value ϵ , KL coefficient β_{KL} , learning rate, rollout batch size, generation settings, and number of PPO minibatch epochs are selected on validation data and reported. After the minibatch updates, the updated policy becomes the next rollout policy, fresh translations are generated from it, and the cycle repeats. PPO needs no within-source group ranking: unlike GRPO, its rollout batch consists of independently rewarded sampled translations.

7.3.3 PPO Inference and Evaluation

After selecting the best PPO checkpoint using validation criteria, inference is exactly like normal translation. The deployed model receives only a new source sentence x and produces one translation with the fixed final decoding configuration. It does not receive y^* , calculate BLEU, ChrF++, COMET, repetition, or length mismatch, or run the reward function at inference time. On the untouched test split, the reference is revealed only after generation to report BLEU, ChrF++, COMET, repetition, and length-mismatch diagnostics. PPO is therefore a pure SFT→RL baseline whose comparison with RECAP-DPO tests whether explicit preference mining contributes beyond direct reward optimization.

8 Experimental Design

For every direction, the main comparison is

$$\boxed{\text{SFT} \quad \text{vs.} \quad \text{DPO}_{\text{Ref}} \quad \text{vs.} \quad \text{RECAP-DPO} \quad \text{vs.} \quad \text{RECAP-DPO} + \text{GRPO}.} \quad (71)$$

Here DPO_{Ref} is a simpler offline baseline whose pairs are ranked by the equal mean of metric-scale-normalized BLEU, ChrF++, and COMET. It omits direction-wise calibration, repetition and length penalties, margin filtering, and balanced pair sampling. RECAP-DPO uses the full fixed reward, pairwise margin filtering, and balanced sampling. If compute permits, SFT→PPO

is added as a pure RL baseline, and SFT→GRPO is reported as an ablation that tests whether DPO initialization matters. The primary success criterion is improvement over the corresponding SFT baseline on held-out BLEU, ChrF++, and COMET, without worsening repetition or length-mismatch measures. Table 7 is the corresponding aggregate held-out report. Any expected ordering is a hypothesis to test, not an assumed result.

8.1 Data Splits and Controlled Comparisons

RECAP treats every translation direction as a separate experiment. For a direction d with N_d source–reference examples, we first shuffle the *sources* once with a recorded random seed and create a fixed 80/10/10 split:

$$\begin{aligned} N_d^{\text{train}} &= \lfloor 0.80N_d \rfloor, \\ \mathcal{T}_d &= \mathcal{T}_d^{\text{train}} \dot{\cup} \mathcal{T}_d^{\text{val}} \dot{\cup} \mathcal{T}_d^{\text{test}}, & N_d^{\text{val}} &= \lfloor 0.10N_d \rfloor, \\ N_d^{\text{test}} &= N_d - N_d^{\text{train}} - N_d^{\text{val}}. \end{aligned} \tag{72}$$

The symbol $\dot{\cup}$ means that the three sets are disjoint: the same source sentence cannot occur in more than one split. For example, if one Bhili direction contains $N_d = 220,364$ source–reference rows, this rule gives 176,291 training sources, 22,036 validation sources, and 22,037 test sources. These counts apply to that one direction only; the other five directions are shuffled and split independently. Thus, Hindi → Bhili and Bhili → Hindi do not share DPO examples merely because they involve the same languages.

What happens to one training source. For each $x_i \in \mathcal{T}_d^{\text{train}}$, the dataset supplies four candidate translations. RECAP computes all four rewards, evaluates all six unordered pairs, retains only pairs whose reward gap exceeds δ , removes duplicates, and applies balanced sampling across generator-pair types. Thus, different training records from the same source can involve Llama–NLLB, mT5–Qwen, or another valid pair, always preserving the higher-reward translation as y^+ .

The maximum number of candidate comparisons is six per source, so

$$N_d^{\text{candidate comparisons}} \leq 6N_d^{\text{train}}, \quad N_d^{\text{DPO}} = |\mathcal{P}_d^{\text{DPO}}| \leq 6N_d^{\text{train}}. \tag{73}$$

For the 220,364-row Bhili-direction example, the 176,291 training sources yield at most 1,057,746 candidate comparisons before margin filtering, deduplication, and balanced sampling. The main DPO run uses all retained balanced pairs from this 80% partition. Smaller source-level subsets are used only for pilot runs and data-size ablations, not because the algorithm requires random subsampling.

Roles of validation and test. The validation 10% is never added to $\mathcal{P}_d^{\text{train}}$. Its BLEU and ChrF++ results determine the common mT5 backbone under the stated macro-average validation rule; it also selects DPO and PPO hyperparameters, including learning rate, early stopping, β , PPO clipping, and KL coefficients. If a separate reward-calibration subset is used, it is drawn only from the training partition and never from validation or test data. Validation translations may be generated to measure performance, but their references are never used for DPO labels, PPO updates, or GRPO rewards.

The test 10% remains untouched until the configuration and checkpoint have been fixed. At that point, the final SFT, DPO, GRPO, and PPO models translate the test sources without seeing their references; BLEU, ChrF++, COMET, repetition, and length-mismatch statistics are then

computed against the held-out references. No test candidate is ranked, filtered, or inserted into DPO training, PPO updates, or GRPO reward calculation. This source-level separation prevents reference, preference-pair, and checkpoint-selection leakage into final evaluation.

All systems should begin from comparable SFT initialization when possible. They should be trained on the same parallel data, evaluated with the same tokenizer and detokenization protocol, and decoded with the same final inference settings. This makes differences easier to attribute to the optimization method rather than to data processing or decoding choices.

For every reported metric, results should include the mean across multiple random seeds and, where feasible, uncertainty estimates such as standard deviation or confidence intervals. Low-resource datasets can be small enough that a few examples strongly influence BLEU or ChrF++; seed variation therefore matters.

8.2 Candidate-Diversity Feasibility Diagnostic

When checkpoint access permits regeneration, a fixed 5K-source training subset can be used to verify that multiple candidate translations contain meaningful diversity. For NLLB, this optional diagnostic compares greedy decoding (beam = 1), beam search with four returned hypotheses, and stochastic sampling with $K = 4$. It reports unique-output ratio, pairwise candidate similarity, BLEU, ChrF++, COMET, quality spread, and oracle improvement. It does not retrain a model or train GRPO; the main pipeline uses the supplied four-model output dataset.

8.3 Preference Construction Ablations

We compare quality-only and full RECAP preferences to identify whether calibration and pathology penalties improve the pseudo-labels. The quality-only component is the equal mean of standardized BLEU, ChrF++, and COMET; the full reward additionally subtracts repetition and length penalties.

The ablation sequence should be cumulative and explicit:

1. **Quality only:** rank with the equal-weight BLEU/ChrF++/COMET quality component.
2. **Quality + repetition:** add only the repetition penalty.
3. **Quality + length:** add only the length-mismatch penalty.
4. **Full reward:** add both repetition and length penalties with the validation-selected convex hyperparameters.
5. **Full reward + margin filtering:** discard pairs whose difference does not exceed δ .
6. **Full RECAP:** combine the full reward, all-pairs construction, margin filtering, disagreement-aware source sampling, and balanced valid-pair sampling.

For each ablation, report the number of retained pairs, average reward margin, source-length distribution, target-length distribution, and candidate-model composition. These statistics help distinguish a true reward improvement from a change in the type or amount of training data; the cumulative result is summarized in Table 8.

Reward-margin distribution template. Plot all candidate-pair margins ΔR before filtering, draw a vertical line at the validation-selected threshold δ, and shade the retained region $\Delta R > \delta$. Illustrative annotation: 1.00M candidate comparisons before filtering; 0.54M retained pairs after filtering and balanced sampling.

Figure 1: Reward-margin distribution before and after filtering. Replace the illustrative counts and threshold with final values.

Model-participation template for retained RECAP pairs.			
	Model	Preferred (C_m^+)	Rejected (C_m^-)
	NLLB	24%	25%
	mT5	31%	20%
	Qwen	18%	38%
	Llama	27%	17%

Figure 2: Grouped bars for preferred and rejected generator participation. Replace the illustrative values with aggregate counts from retained pairs.

8.4 Automatic Preference Consistency and Model Participation

Because human pairwise labels are unavailable, RECAP reports two automatic consistency diagnostics rather than claiming direct human validation. First, *preference agreement* is the fraction of retained pairwise preferences whose orientation agrees with the quality-only ranking based on BLEU, ChrF++, and COMET. Second, Spearman’s ρ measures agreement between the full RECAP ranking and the quality-only ranking of all four candidates for a source. These diagnostics indicate how much the penalties and calibration alter metric-based rankings; they are not substitutes for human judgments.

The same subsection reports C_m^+ and C_m^- for every candidate model. Together with pair-type frequencies, these quantities show whether RECAP draws on heterogeneous candidate strengths or collapses into a nearly fixed teacher–student pattern.

8.5 Model-Diversity Ablation

We compare candidates sampled from one SFT model,

$$\mathcal{V}_{\text{self}}(x) = \{y_1, \dots, y_K\}, \quad y_i \sim \pi_{\text{SFT}}, \quad (74)$$

with candidates from heterogeneous models:

$$\mathcal{V}_{\text{hetero}}(x) = \{y^{\text{NLLB}}, y^{\text{mT5}}, y^{\text{Qwen}}, y^{\text{Llama}}\}. \quad (75)$$

This tests whether model diversity provides more useful preference supervision than sampling diversity.

To make this test fair, the two settings should use the same number of candidates per source and approximately the same number of final preference pairs. The self-sampling baseline can use multiple temperatures or beams, but it should not receive more compute or more candidates than the heterogeneous setting. A useful diagnostic is the distribution of pairwise reward margins: if heterogeneous models produce larger and more varied margins, this supports the claim that their diversity creates better preference signals. Table 9 reports this controlled comparison.

8.6 GRPO Data Size and Group-Size Ablations

GRPO is more expensive than offline DPO because it generates fresh translations at every rollout. Therefore, each direction uses a fixed source-level GRPO subset from its own training split. The initial main setting is 50K sources per direction when that many are available; no sources are pooled across languages or directions. With $G = 2$ or $G = 3$, this requires 100K or 150K generated candidates per rollout, respectively, for one 50K-source direction.

The data-size ablation uses 10K, 25K, and 50K source subsets. The group-size ablation uses

$$G \in \{2, 3\} \tag{76}$$

on the same fixed source subset. The main paper reports both group sizes and their trade-off because group size is central to GRPO. Detailed data-size curves, compute cost, and additional decoding settings are reported in the appendix. Every comparison reports held-out BLEU, ChrF++, COMET, repetition, and length mismatch; a larger reward alone is not evidence of a better translation system.

8.7 Evaluation Protocol and Success Criteria

Automatic evaluation reports BLEU, ChrF++, and COMET as three separate primary translation-quality metrics, together with repetition rate and length ratio as degeneration diagnostics. The three quality metrics are never collapsed into a single final test number. Two useful diagnostics are

$$\rho_{\text{len}} = \frac{|y|}{|y^*|}, \quad \rho_{\text{rep}} = \frac{N_{\text{repeated ngrams}}}{N_{\text{ngrams}}}. \tag{77}$$

The held-out evaluation uses the same fixed metric configurations as training-time scoring, but reports the metrics separately. It does not claim that these automatic values constitute a complete assessment of translation quality.

8.7.1 Evaluation Metrics and Reporting

BLEU and ChrF++ provide complementary reference-fidelity signals: BLEU focuses on n-gram precision, while ChrF++ uses character and word n -grams and can be more tolerant of morphology and spelling variation. COMET provides a learned reference-aware quality signal and is reported with its fixed checkpoint and score direction. Repetition rate and length ratio are reported separately because an aggregate lexical score can hide degenerate repetition or unusually short and long output.

Results should be presented per direction as well as averaged across directions. An unweighted average gives every direction equal importance, while a corpus-size-weighted average can be dominated by a larger direction. Reporting both, with the aggregation rule stated clearly, gives a more complete picture. The aggregate metrics belong in Table 7, while Table 10 audits whether the automatically constructed preferences agree with a small human-evaluated subset.

For every direction d , the main reported quantities should be the change from SFT:

$$\Delta\text{BLEU}_d = \text{BLEU}(\text{RECAP}_d) - \text{BLEU}(\text{SFT}_d), \tag{78}$$

and

$$\Delta\text{ChrF}++_d = \text{ChrF}++(\text{RECAP}_d) - \text{ChrF}++(\text{SFT}_d). \tag{79}$$

Direction-wise RECAP-DPO improvement over SFT template.				
	Direction	Δ BLEU	Δ ChrF++	Δ COMET
	Hi→Bhi	+2.1	+2.8	+0.021
	Bhi→Hi	+2.7	+3.4	+0.027
	Hi→Gon	+3.1	+3.8	+0.031
	Gon→Hi	+2.4	+3.0	+0.024
	Hi→Mun	+2.9	+3.5	+0.029
	Mun→Hi	+2.3	+3.2	+0.026

Figure 3: Six-direction heatmap specification for RECAP-DPO minus SFT. Replace the illustrative values with final held-out deltas.

and

$$\Delta\text{COMET}_d = \text{COMET}(\text{RECAP}_d) - \text{COMET}(\text{SFT}_d). \quad (80)$$

The framework is successful only when these changes are positive overall and are not achieved by increasing repetition or length mismatch.

8.8 Reward-Hacking Analysis

Metric-based RL can suffer from reward hacking: an automatic score may increase while human-perceived translation quality decreases:

$$R_{\text{automatic}} \uparrow \quad \text{while} \quad Q_{\text{human}} \downarrow. \quad (81)$$

We will inspect cases in which held-out reference metrics and the reward components give a conflicting picture. In particular, we will look for repetition, excessive copying, empty output, and unusually short or long output.

For each failure category, the analysis should include the source sentence, reference, output, reward-component values, and a short explanation. Examples should cover both false positives, where the reward scores a bad translation highly, and false negatives, where the reward rejects a translation that humans consider good. This analysis is essential for deciding whether a reward component needs a different weight, a better detector, or removal.

8.9 Reproducibility and Reporting

RECAP experiments should report the SFT base model, model size, tokenizer, parallel-data source, split sizes, data cleaning rules, hardware, training duration, optimizer, learning-rate schedule, batch size, sequence lengths, decoding configuration, and random seeds. For preference mining, report the candidate models, the number of candidates per source, reward weights, calibration set, normalization method, confidence threshold, disagreement rule, number of pairs before and after filtering, and the distribution of margins.

For DPO, GRPO, and PPO, report the reference policy, β , KL coefficient, number of updates, group size, clipping parameter, generation settings, and early-stopping rule. A released preference dataset should include enough metadata to reproduce a preference label. If translation data cannot be shared, aggregate statistics and a documented generation script should still be provided. The required appendix analyses are listed in Table 11.

9 Limitations

RECAP depends on the quality of its reference-based reward. BLEU, ChrF++, and COMET can disagree with one another and with qualified human judgments, especially when a learned evaluator has limited coverage for the languages in scope. The repetition and length penalties detect only a narrow set of observable pathologies; they do not establish semantic adequacy, fluency, language identity, or absence of hallucination. Consequently, pseudo-preferences may still favour similarity to a single reference. Heterogeneous candidates and online GRPO rollouts also increase compute cost. Direction isolation improves experimental control but does not exploit possible transfer between related directions.

These limitations do not invalidate the framework. They define the conditions under which its results should be interpreted: RECAP is a method for testing whether better pseudo-preference construction helps low-resource NMT, not a replacement for broad and careful human evaluation.

10 Conclusion

RECAP treats each translation direction as an independent learning problem. It starts from stored heterogeneous SFT-finetuned candidate outputs, uses a calibrated reward that combines BLEU, ChrF++, COMET, repetition, and length mismatch, and applies confidence plus disagreement to select useful preference pairs. These pairs train DPO from the existing direction-specific mT5 SFT checkpoint, while GRPO can provide further on-policy refinement and PPO remains a controlled RL baseline. The central outcome is not preference construction alone: it is whether this framework can improve held-out BLEU, ChrF++, and COMET over SFT without increasing observable repetition or length mismatch when data and human annotations are scarce.

11 Instructions to Coding Agent

This is the executable specification for the experiments in this paper. Implement the pipeline to answer one question: can confidence-filtered, direction-calibrated pseudo-preferences from the *stored* outputs of SFT-finetuned NLLB, mT5, Qwen, and Llama improve the corresponding mT5 SFT model? Preserve the difference between static candidates used by offline DPO and fresh on-policy samples required by GRPO and PPO.

11.1 A. Required architecture and invariants

Implement one independent run per direction $d \in \mathcal{D}$. Each run owns its data manifest, training, validation, and test splits, metric configuration, calibration statistics, reward weights, preference files, mT5 checkpoint, frozen reference checkpoint, optimizer state, logs, and final report. Never pool examples, z-score statistics, DPO pairs, model parameters, or RL trajectories across directions.

The existing four outputs are static records:

$$\{y_i^{(\text{NLLB})}, y_i^{(\text{mT5})}, y_i^{(\text{Qwen})}, y_i^{(\text{Llama})}\}. \quad (82)$$

Do not retrain NLLB, Qwen, or Llama merely to generate these candidates. Use the existing direction-specific mT5 SFT checkpoint as the common trainable policy $\pi_{\text{SFT},d}$ for DPO, PPO, and optional GRPO. Whenever a trainer requires a reference policy, clone and freeze the correct starting checkpoint; no reference-policy parameter may receive gradients.

References are supervision for scoring only. The translation model receives source text and its normal generation prompt, never the gold translation. This rule applies to SFT evaluation, DPO validation decoding, PPO rollouts, GRPO rollouts, and final test decoding.

11.2 B. Data manifest, split, and audit files

First construct a source-level manifest for each direction. Assign a stable identifier to every source/reference record and split sources into train, validation, and test sets before pair construction. Use a fixed source-level 80%/10%/10% split unless a documented project split supersedes it. Exact duplicates, normalized duplicates, and near duplicates must stay in the same split. Store source identifier, source checksum, reference checksum, direction, split, candidate-column mapping, and split seed.

Only training sources can be used to fit calibration statistics and construct DPO pairs. Validation sources are for predeclared hyperparameter and checkpoint selection by decoding the model; they do not contribute DPO pairs. Test sources are sealed until final evaluation. If reference-aware reward hyperparameters need a calibration subset, reserve a fixed subset from training before pair mining or use a nested training fold. Never use validation or test references to create preference labels.

Every loaded row must retain:

1. source x_i , reference y_i^* , direction, split, and source identifier;
2. all four stored candidate texts and their generator labels;
3. stored BLEU and ChrF++ values, original scale, and their validation status;
4. COMET, repetition, length, standardized components, reward, pair decisions, and exclusion flags once computed.

Reject and log rows with missing source/reference text, invalid candidate mapping, empty or malformed candidates, non-finite metrics, or incompatible metric configuration. Do not silently discard records. Normalize whitespace consistently for metric calculation and duplicate detection, but retain the original completion text used for training unless the normalization belongs to the documented tokenizer pipeline.

11.3 C. RewardEngine: metric calculation and calibration

Implement a direction-aware **RewardEngine** with three explicit responsibilities:

1. `fit(training_candidates)` validates metric scales, computes all raw training signals, and saves direction-specific calibration parameters;
2. `score_batch(sources, candidates, references)` returns raw metrics, standardized metrics, penalty values, final reward, and validity flags for each completion;
3. `serialize()` saves checkpoint identifiers, tokenization/smoothing configuration, calibration values, hyperparameters, and a configuration hash.

Use one fixed sentence-level BLEU configuration and one fixed ChrF++ configuration throughout the study. Convert verified stored BLEU and ChrF++ values from $[0, 100]$ to $[0, 1]$ before calibration. If stored scores cannot be verified as comparable, recompute them using the candidate and reference with one documented configuration. Compute COMET in batches with a fixed

reference-aware checkpoint and the ordered triple (x, y, y^*) . Check score orientation on a small sanity set and record the model/checkpoint version.

For each completion calculate $B_d(y, y^*)$, $H_d(y, y^*)$, $C_d(x, y, y^*)$, $P_{\text{rep},d}(y)$, and $P_{\text{len},d}(y, y^*)$. The repetition penalty must use a documented tokenization and repeated- n -gram rule. The length penalty must be stable for short strings, such as a clipped absolute log-ratio of candidate and reference length. Empty output, extreme length, non-finite scores, and unsupported characters should create explicit flags and cannot accidentally receive a high reward.

Fit each standardization parameter only on training candidates of the same direction:

$$\tilde{q}_d = \frac{q - \mu_{q,d}^{\text{train}}}{\max(\sigma_{q,d}^{\text{train}}, \epsilon)}. \quad (83)$$

Save these values and apply them unchanged to DPO data, validation decoding, PPO rollouts, GRPO rollouts, and final test analysis. Never refit the calibrator on a rollout group, validation set, or test set. Define

$$L_{\text{quality},d}(x, y, y^*) = \frac{\tilde{B}_d(y, y^*) + \tilde{H}_d(y, y^*) + \tilde{C}_d(x, y, y^*)}{3}, \quad (84)$$

and

$$R_{\text{RECAP},d}(x, y, y^*) = w_{\text{quality}}L_{\text{quality},d} - w_{\text{rep}}\tilde{P}_{\text{rep},d} - w_{\text{len}}\tilde{P}_{\text{len},d}. \quad (85)$$

The reward weights are fixed, validation-selected hyperparameters:

$$w_{\text{quality}}, w_{\text{rep}}, w_{\text{len}} \geq 0, \quad w_{\text{quality}} + w_{\text{rep}} + w_{\text{len}} = 1. \quad (86)$$

Search a predeclared simplex grid, select only through permitted training-calibration and validation procedures, then freeze the selected values for final testing. The example (0.60, 0.25, 0.15) is illustrative, not a mandatory hard-coded setting. This is an automatic reward function, not a separately trained neural reward model.

11.4 D. PreferenceBuilder: create fixed DPO pairs

Implement **PreferenceBuilder** using only training sources, static stored outputs, and the fitted **RewardEngine**. Score all four candidates for a source, deduplicate normalized-identical completions while preserving an audit list of their generator labels, enumerate up to six unordered pairs, and orient a valid pair as (y^+, y^-) only if $R(y^+) > R(y^-)$.

Retain a pair only when it is valid, non-duplicate, belongs to training, passes the configured safety rules, and satisfies

$$R(y^+) - R(y^-) \geq \delta_d. \quad (87)$$

Select δ_d before final testing and report its value, retention rate, and reward-margin distribution. Limit or reweight the number of pairs per source so a few sources do not dominate. Balance pair types across generator identities and reward-margin bins where data allow; otherwise DPO may simply imitate whichever stored generator wins most often.

Write immutable per-direction pair files. A record must include prompt/source, chosen and rejected completion, source identifier, selected generator labels, raw metric vectors, standardized components, final rewards, reward margin, filter decisions, split, random seed, and configuration hash. Save rejected pairs with reasons. DPO reads these fixed files; it does not rescore candidates on the fly.

11.5 E. Implementation reconnaissance: explore and adapt existing code

Before writing a custom training loop, the coding agent must actively inspect maintained trainer implementations and translation-specific research code. The preferred implementation path is the maintained Hugging Face TRL repository: use its standard `DPOTrainer`, `PPOTrainer`, and `GRPOTrainer` whenever their current pinned version supports the mT5 architecture and the required RECAP hooks. Pin the exact release or commit hash that will be used; trainer APIs and argument names can change, so do not implement against an unpinned online example.

Use the original DPO reference implementation to verify the pairwise log-probability calculation and loss sign from [7]. Also inspect the official DPO-MBR implementation accompanying translation-focused DPO work [14]. Compare these with the translation preference and reward-design literature already cited in this paper, especially [1, 12], and with RL-for-MT practice in [11, 13]. For GRPO, use the original algorithmic description [10]; for PPO, verify the implementation against [9].

Create a short implementation-reconnaissance note before the first full run. It must record repository URLs, commit hashes or release versions, licenses, relevant trainer example files, dataset field expectations, sequence-model assumptions, and the exact RECAP changes. In particular, verify that the selected trainer version correctly handles the encoder-decoder mT5 architecture. A causal-LM example must not be copied unchanged into mT5 training. Test one forward/backward batch and one generation batch before scaling up.

Adapt existing Hugging Face code rather than rewriting stable infrastructure: retain the trainer’s distributed training, checkpointing, gradient accumulation, mixed precision, and optimizer handling where compatible. Replace or wrap only the RECAP-specific pieces: source-to-prompt formatting, fixed offline pair ingestion, direction-specific reward calibration, static-candidate auditing, the GRPO reward callback, PPO terminal reward injection, validation decoding, and required logging. If a standard trainer cannot support encoder-decoder mT5 or a required RECAP hook, record the concrete incompatibility and implement only the smallest documented adapter or wrapper needed to resolve it. Document every divergence from upstream behavior, including reward normalization, reference-policy creation, padding/masking, encoder-decoder log-probability computation, and checkpoint selection. Never copy repository defaults without checking that they satisfy the split, reference-access, and online-versus-offline constraints in this section.

11.6 F. DPOTrainer integration

Implement the offline optimization stage with `DPOTrainer`, or a thin adapter that preserves its standard prompt/chosen/rejected semantics. For direction d , initialize both models from the same existing mT5 SFT checkpoint:

$$\pi_{\theta,d} \leftarrow \pi_{\text{SFT},d}, \quad \pi_{\text{ref},d} \leftarrow \text{freeze}(\pi_{\text{SFT},d}). \quad (88)$$

The adapter must map a fixed pair record to the trainer’s prompt, chosen, and rejected fields. Policy and frozen reference must use the same tokenizer, source template, maximum source/target length, special tokens, and truncation policy.

The DPO objective is

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E} \left[\log \sigma \left(\beta \left[\log \frac{\pi_{\theta}(y^+ | x)}{\pi_{\text{ref}}(y^+ | x)} - \log \frac{\pi_{\theta}(y^- | x)}{\pi_{\text{ref}}(y^- | x)} \right] \right) \right]. \quad (89)$$

Expose β , learning rate, batch size, gradient accumulation, warmup, maximum sequence length, number of epochs or steps, gradient clipping, checkpoint frequency, and validation-selection rule

in the resolved configuration. The frozen reference is immutable. Select a DPO checkpoint by the predeclared validation rule, for example a composite of BLEU, ChrF++, and COMET subject to repetition and severe-length constraints, rather than by training loss or test performance.

At every reporting interval log DPO loss, preference accuracy, chosen and rejected log probabilities, implicit KL/reference divergence, gradient norm, learning rate, token count, skipped examples, and completion lengths. Decode the validation sources at fixed checkpoints using a fixed generation configuration and save the selected checkpoint, tokenizer, trainer state, reference-checkpoint identifier, pair-file checksum, and source-manifest checksum.

Use matched splits and approximately matched optimization budgets for these offline conditions:

1. **SFT**: untouched direction-specific mT5 SFT checkpoint;
2. **DPO–raw**: pairs oriented by uncalibrated raw metric aggregation;
3. **DPO–quality-only**: calibrated BLEU, ChrF++, and COMET without repetition or length penalties;
4. **DPO–no-confidence**: full reward but no reward-margin filtering;
5. **RECAP-DPO**: full calibrated reward, confidence filtering, deduplication, and balanced heterogeneous pairs.

11.7 G. GRPOTrainer integration

Implement online refinement with **GRPOTrainer**, or a compatible wrapper. The stored NLLB, mT5, Qwen, and Llama strings are not valid GRPO rollouts. Each GRPO update must call the current trainable policy’s generation method and score those fresh completions.

For the proposed RECAP-DPO+GRPO condition, initialize the trainable policy from the selected RECAP-DPO checkpoint and freeze a copy of that DPO checkpoint as the KL reference. For the pure-GRPO ablation, initialize the policy and frozen KL reference from the original direction-specific mT5 SFT checkpoint. Configure stochastic generation and sample $G \in \{2, 3\}$ translations for each training source. The trainer’s reward callback must receive source, completion, reference, and direction identifier, call the frozen **RewardEngine**, and return one scalar reward plus a structured diagnostic record per completion.

For rewards $\{r_{i,g}\}_{g=1}^G$ from source x_i , calculate a group-relative advantage, for example

$$A_{i,g} = \frac{r_{i,g} - \bar{r}_i}{\sqrt{\frac{1}{G} \sum_{g'=1}^G (r_{i,g'} - \bar{r}_i)^2 + \epsilon}}, \quad \bar{r}_i = \frac{1}{G} \sum_{g=1}^G r_{i,g}. \quad (90)$$

If the group standard deviation is effectively zero, set all group advantages to zero or skip the group using one documented deterministic rule. Optimize the trainer’s clipped group-relative objective with KL regularization to the frozen reference. Verify in a test run that groups never mix sources, each completion is scored individually, and the static candidate files are never read as rollout actions.

Log mean, standard deviation, minimum, and maximum reward; all reward components; advantage statistics; KL; clipping statistics; generated length; repetition; invalid-output count; temperature; and zero-variance-group rate. Decode validation sources at fixed intervals and select checkpoints under the same validation rule as DPO. The group-size ablation compares $G = 2$ and $G = 3$ using the same source-update budget; report source groups, total generated completions, wall-clock cost, and quality/safety outcomes.

11.8 H. PPOTrainer integration

Implement **PPOTrainer** as an independent RL baseline. Initialize PPO from the direction-specific mT5 SFT checkpoint, freeze an SFT copy for KL control, and attach the value head required by the selected PPO implementation. PPO does not begin from DPO in the primary comparison; its role is to test conventional reward optimization from SFT.

For each training source, generate a new completion from the current PPO policy, compute its terminal $R_{\text{RECAP},d}(x, y, y^*)$, combine it with the configured per-token KL cost relative to the frozen SFT reference, compute returns and advantages with documented GAE parameters, and make clipped PPO updates. Use the same reward engine, source split, tokenizer, output limits, metric configuration, and validation protocol as GRPO. Do not train a separate reward model and do not use static candidate strings as PPO actions.

Log policy loss, value loss, total loss, entropy, mean terminal reward, reward components, approximate KL, KL coefficient, clip fraction, value clip fraction, explained variance, gradient norm, generated length, repetition, and invalid-output rate. Preserve evidence of instability. Only predeclared safeguards, such as a KL schedule, early stopping, or maximum-length rule, may be applied; never tune those safeguards using test outcomes.

11.9 I. Experiment and ablation matrix

Use the same source split, fixed test set, direction-specific mT5 backbone, and evaluation setup for all methods. Run multiple random seeds for learning conditions whenever possible and report seed mean and uncertainty, not only the best run. The minimum core matrix is:

1. SFT;
2. DPO–raw;
3. DPO–quality-only;
4. DPO–no-confidence;
5. RECAP-DPO;
6. SFT→PPO;
7. SFT→GRPO, when compute permits;
8. SFT→RECAP-DPO→GRPO.

Exactly what differs among the DPO variants. All four DPO conditions use the same direction-specific mT5 SFT initialization, frozen SFT reference, stored four-model candidate pool, source split, DPO objective, tokenizer, decoding/evaluation protocol, and approximately matched update budget. They differ *only* in how the offline better–worse pairs are scored and selected:

DPO–raw

Rank candidates by a simple uncalibrated equal average of BLEU, ChrF++, and COMET after only the required metric-scale conversion. It has no direction-wise z -score calibration, no repetition or length penalty, no reward-margin confidence filter, and no balanced pair sampling. This is the simple reference-pair baseline.

DPO-quality-only

Use direction-calibrated BLEU, ChrF++, and COMET to form L_{quality} , but set the repetition and length-penalty weights to zero. Keep RECAP’s deduplication, margin filtering, and pair balancing. The difference from full RECAP-DPO therefore isolates whether the two output-quality penalties help.

DPO-no-confidence

Use the full direction-calibrated RECAP reward, including repetition and length penalties, together with deduplication and pair balancing, but retain every valid strictly ordered pair by setting the reward-margin threshold to zero. The difference from full RECAP-DPO isolates the value of rejecting low-margin, uncertain pseudo-preferences.

RECAP-DPO

Use the complete pipeline: direction-calibrated quality metrics, repetition and length penalties, deduplication, a validation-selected positive reward-margin threshold, confidence weighting when configured, and balanced heterogeneous pair sampling.

Run these targeted ablations:

1. **Candidate diversity:** heterogeneous stored pool versus repeated samples from mT5 SFT, if mT5 generation access is available. Match candidate count, decoding budget, and retained pair count as closely as possible.
2. **Calibration:** raw aggregation, global normalization, and direction-specific training-only calibration.
3. **Reward components:** quality only; quality plus repetition penalty; quality plus length penalty; and full reward.
4. **Confidence:** no margin threshold and a small predeclared grid of δ_d thresholds; report retained-pair count and margin distribution.
5. **Pair selection:** all valid pairs, one best-worst pair per source, and balanced multi-pair sampling, with total optimization examples matched where feasible.
6. **Data scale:** predeclared source budgets such as 10K, 25K, and 50K when available; report actual sources and pairs per direction.
7. **GRPO group size:** $G = 2$ versus $G = 3$ with matched source-update budget.
8. **Weight sensitivity:** selected reward weights versus neighboring simplex-grid values. Only the validation-selected setting is eligible for the final primary test claim.

The candidate-feasibility diagnostic is separate from RL. On a fixed training-only subset, report unique-output ratio, pairwise similarity, sentence BLEU, ChrF++, COMET, reward spread, and oracle improvement for the heterogeneous static pool and, if feasible, mT5 self-sampling. Oracle improvement is the gain available if an oracle could choose the best candidate for each source; it is an upper-bound diagnostic, not a deployable result.

11.10 J. Evaluation protocol and statistical reporting

For every selected checkpoint and direction, decode every held-out test source with the same pre-declared decoding configuration. The model receives the source only. Compute corpus BLEU, corpus ChrF++, corpus COMET, average sentence-level quality scores, repetition rate, severe-length-mismatch rate, invalid-output rate, and average output length. Report absolute values and deltas from the corresponding mT5 SFT baseline.

Show all six direction-level results before any macro average. Macro-average across directions only after each direction has equal weight; do not let a larger corpus determine the conclusion. Keep source-level predictions for paired comparison. Use paired bootstrap confidence intervals or another documented paired significance procedure for key changes from SFT. A successful method improves held-out translation quality without a material rise in repetition, severe length mismatch, invalid output, or reward-hacking indicators. If metrics disagree, report the trade-off rather than selecting only the favorable metric.

11.11 K. Mandatory tables, plots, and loss curves

Write machine-readable result files and produce paper-ready artifacts for each direction and each method:

1. a candidate-pool table with model-level BLEU, ChrF++, COMET, repetition, length mismatch, candidate count, unique-output ratio, reward mean/spread, and selected-pair participation;
2. a main-results table for SFT, DPO baselines, RECAP-DPO, PPO, pure GRPO when run, and RECAP-DPO+GRPO, including BLEU, ChrF++, COMET, repetition, severe length mismatch, and macro average;
3. a reward/pair-ablation table with retained pairs, mean margin, quality metrics, repetition, and length mismatch;
4. a diversity table comparing heterogeneous static candidates with repeated mT5 samples under matched budgets;
5. a heat map of metric deltas from SFT by direction and method;
6. a reward-margin histogram or density plot before and after confidence filtering;
7. a model-participation plot showing which generators most often provide chosen and rejected completions;
8. DPO curves for loss, preference accuracy, chosen/rejected log probabilities, implicit KL, validation BLEU, validation ChrF++, validation COMET, repetition, and length mismatch versus update;
9. GRPO curves for mean reward, component rewards, advantage standard deviation, KL, generated length, repetition, invalid-output rate, and validation metrics versus update;
10. PPO curves for policy loss, value loss, entropy, terminal reward, approximate KL, clip fraction, explained variance, output length, repetition, and validation metrics versus update;
11. reward-weight sensitivity and GRPO group-size comparison plots;

12. a qualitative-error table with source, reference, SFT output, RECAP-DPO output, optional GRPO/PPO output, reward components, and short error label. Select examples by a documented sampling rule, not by cherry-picking.

Do not reduce learning behavior to final numbers alone. Preserve raw step-level trainer logs so reward hacking, collapse, instability, or divergence can be inspected. Every figure must identify direction, method, seed aggregation rule, metric scale, and whether it shows train, validation, or test data.

11.12 L. Automated checks and failure handling

Implement tests before expensive runs:

1. reward weights are non-negative and sum to one within numerical tolerance;
2. holding quality fixed, increasing repetition or length mismatch cannot increase reward;
3. all standardized quantities use saved training-only statistics and are finite;
4. identical completions never form a DPO pair and every retained pair has positive reward margin;
5. no source identifier occurs in more than one split and no validation/test row enters pair mining;
6. DPO, PPO, and GRPO reference-policy parameters are unchanged;
7. GRPO and PPO completions are generated by the current policy at update time rather than loaded from static candidate files;
8. each rollout reward receives the correct source, reference, candidate, and direction;
9. every final checkpoint, manifest, pair file, calibration file, metric configuration, seed, and trainer state is recorded with a hash.

When a direction has missing checkpoint access, insufficient valid pairs, unsupported COMET coverage, metric failure, or unstable RL, write a direction-specific failure report containing the relevant counts, component distributions, logs, and attempted predeclared safeguards. Do not substitute data, statistics, or model results from a different direction.

11.13 M. Reproducibility deliverables and completion criterion

Implement the project as a small, configuration-driven codebase. Use one primary command-line entry point that accepts an experiment configuration and optional overrides; do not create separate launch scripts or copied training loops for DPO, PPO, GRPO, or each ablation. Keep the file layout minimal: one entry point, a small number of reusable modules for data/preference construction, reward calculation, trainer adaptation, and evaluation, plus configuration files. Split a module only when it has a distinct reusable responsibility. The coding agent may choose file names and directories, but must avoid duplicated logic, notebook-only execution, hidden global state, and manually edited experiment settings.

Every experiment must be reproducible from a single configuration file. A configuration must specify at least: direction and data selection; split seed; base mT5 checkpoint; trainer mode (SFT, DPO, PPO, or GRPO); initialization and frozen-reference checkpoint; static-candidate-pool mode; metric and COMET configuration; calibration mode and saved-statistics source; reward weights

and enabled penalties; margin threshold; pair-selection and balancing policy; DPO/PPO/GRPO hyperparameters; PPO KL and GAE settings; GRPO group size; decoding settings; validation-selection metric; random seeds; logging cadence; and output identifier.

Define ablations as configuration overrides, not new code paths. For example, a single base configuration should be able to select raw versus calibrated reward, enable or disable repetition and length penalties, alter reward weights, change confidence filtering, choose pair-selection strategy, switch heterogeneous versus mT5-only candidates, set DPO/PPO/GRPO mode, select pure GRPO versus DPO+GRPO initialization, change G , and reduce training-data scale. A sweep configuration should expand these named overrides into a deterministic experiment matrix and assign each resolved run a unique identifier. The runtime must write the fully resolved configuration beside the outputs before training begins.

For every run, save the resolved configuration, exact command, code revision hash, package/environment versions, hardware and precision settings, all random seeds and deterministic-mode settings, data-manifest checksum, tokenizer and checkpoint identifiers, reward-engine serialization, pair files, rejection audit, training logs, validation-selection decision, final test predictions, aggregate metrics, and plotting inputs. Save enough intermediate state to regenerate every paper table and figure without rerunning optimization.

The implementation is complete only when a fresh user can reproduce any reported run from its saved configuration and one documented command, create audited fixed DPO pairs from stored four-model outputs, train the specified `DPOTrainer`, `PPOTrainer`, and `GRPOTrainer` conditions from the correct mT5 checkpoints, evaluate them without test leakage, and emit all required tables, figures, loss curves, checkpoints, and metadata. The final experimental report must state actual hyperparameters, direction availability, candidate/pair counts, excluded rows, seeds, and every deviation from this specification.

12 References

References

- [1] Guofeng Cui, Pichao Wang, Yang Liu, Zemian Ke, Zhu Liu, and Vimal Bhat. 2025. CRPO: Confidence-Reward Driven Preference Optimization for Machine Translation. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 560–574.
- [2] Júlia Falcão, Claudia Borg, Nora Aranberri, and Kurt Abela. 2024. COMET for Low-Resource Machine Translation Evaluation: A Case Study of English–Maltese and Spanish–Basque. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation*, pages 3553–3565.
- [3] Zhaopeng Feng, Shaosheng Cao, Jiahan Ren, Jiayuan Su, Ruizhe Chen, Yan Zhang, Jian Wu, and Zuozhu Liu. 2025. MT-R1-Zero: Advancing LLM-based Machine Translation via R1-Zero-like Reinforcement Learning. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 18685–18702.
- [4] Barry Haddow, Rachel Bawden, Antonio Valerio Miceli Barone, Jindřich Helcl, and Alexandra Birch. 2022. Survey of Low-Resource Machine Translation. *Computational Linguistics*, 48(3):673–732.
- [5] Paul Nganga Kamau. 2025. Challenge Track: Breaking Language Barriers: Adapting NLLB-200 and mBART for Bhili, Gondi, Mundari, and Santali Without Source Language Proficiency.

- In *Proceedings of MMLoSo 2025: First Workshop on Multimodal Models for Low-Resource Contexts and Social Impact*, pages 106–108.
- [6] Hellina Hailu Nigatu, Atnafu Lambebo Tonja, Benjamin Rosman, Thamar Solorio, and Monojit Choudhury. 2024. The Zeno’s Paradox of “Low-Resource” Languages. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 17753–17774.
 - [7] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems*, volume 36.
 - [8] Ricardo Rei, Craig Stewart, Ana C. Farinha, and Alon Lavie. 2020. COMET: A Neural Framework for MT Evaluation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 2685–2702.
 - [9] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347.
 - [10] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv preprint arXiv:2402.03300.
 - [11] Shiqi Shen, Yong Cheng, Zhongjun He, Wei He, Hua Wu, Maosong Sun, and Yang Liu. 2016. Minimum Risk Training for Neural Machine Translation. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1683–1692.
 - [12] Kaden Uhlig, Joern Wuebker, Raphael Reinauer, and John Denero. 2025. Cross-lingual Human-Preference Alignment for Neural Machine Translation with Direct Quality Optimization. In *Proceedings of the Tenth Conference on Machine Translation*, pages 31–51.
 - [13] Lijun Wu, Fei Tian, Tao Qin, Jianhuang Lai, and Tie-Yan Liu. 2018. A Study of Reinforcement Learning for Neural Machine Translation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3612–3621.
 - [14] Guangyu Yang, Jinghong Chen, Weizhe Lin, and Bill Byrne. 2024. Direct Preference Optimization for Neural Machine Translation with Minimum Bayes Risk Decoding. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)*, pages 391–398.

Table 1: Self-contained comparison of RECAP with representative related approaches. The distinction is the full RECAP combination, not any one feature in isolation.

Approach	What it mainly uses or optimizes	How RECAP differs
Low-resource SFT and multilingual adaptation [5]	Fine-tuning on parallel source-reference pairs.	RECAP starts from the same SFT baseline, then creates better-worse translation comparisons and tests whether post-training improves held-out metrics.
Minimum-risk training and policy-gradient MT [11, 13]	Direct optimization of sequence-level loss or reward from model samples.	RECAP converts candidate quality into confidence-filtered pseudo-preferences and uses a direction-calibrated reward with explicit reference-fidelity, repetition, and length-mismatch signals.
Generic DPO and GRPO [7, 10]	Generic preference pairs (DPO) or on-policy groups with rewards (GRPO).	RECAP supplies the MT-specific data construction: heterogeneous translations for each source, calibrated translation-quality scoring, confidence filtering, and low-resource error safeguards.
MBR-derived DPO for MT [14]	Preferences obtained from Minimum Bayes Risk decoding and distilled with DPO.	RECAP ranks heterogeneous model outputs sentence by sentence and isolates the contribution of calibration, confidence filtering, and candidate disagreement.
Quality-estimation alignment / DQO [12]	A quality-estimation proxy is used as the preference signal.	RECAP uses one fixed reference-aware COMET signal alongside BLEU, ChrF++, repetition, and length checks; it does not train a new quality estimator or treat COMET as human ground truth.
CRPO [1]	Samples candidates from one reference policy and selects difficult pairs using reward difference plus policy-likelihood confidence.	RECAP uses heterogeneous candidate generators and reward-margin <i>label</i> confidence. It tests direction calibration, error-aware scoring, and disagreement; CRPO remains a direct baseline.
RECAP (this work)	Heterogeneous candidates for each source plus its reference during training; offline DPO and optional online GRPO.	Tests the full direction-isolated pipeline across six Hindi↔Bhili/Gondi/Mundari directions: candidate diversity → calibrated multi-aspect reward → reliable preferences → improvement over SFT without more translation errors.

Table 2: Schema of each direction-specific CSV file in the provided dataset.

File	Direction	Columns in each row
hi2tgt.csv	Hindi → target	Same structure, with target-language candidate columns.
tgt2hi.csv	Target → Hindi	Same structure, with Hindi_nllb, Hindi_mt5, Hindi_qwen, and Hindi_llama.

Table 3: Reference-model performance for Bhili. Each cell reports BLEU / ChrF++.

Direction	Model	Validation	Test
Hi → Bhi	NLLB	23.82 / 50.59	24.20 / 50.94
Hi → Bhi	mT5	27.59 / 53.31	27.68 / 53.27
Hi → Bhi	Qwen	18.39 / 43.63	18.40 / 43.60
Hi → Bhi	Llama	32.48 / 56.04	32.51 / 55.95
Bhi → Hi	NLLB	54.27 / 70.54	55.42 / 71.30
Bhi → Hi	mT5	62.36 / 76.05	62.61 / 76.18
Bhi → Hi	Qwen	41.35 / 59.59	41.22 / 59.36
Bhi → Hi	Llama	61.07 / 75.12	61.40 / 75.19

Table 4: Reference-model performance for Gondi. Each cell reports BLEU / ChrF++.

Direction	Model	Validation	Test
Hi → Gon	NLLB	19.13 / 43.71	18.18 / 42.89
Hi → Gon	mT5	26.96 / 49.04	26.08 / 48.33
Hi → Gon	Qwen	16.96 / 41.95	16.61 / 41.19
Hi → Gon	Llama	21.61 / 45.73	20.30 / 44.64
Gon → Hi	NLLB	37.23 / 56.11	36.85 / 56.02
Gon → Hi	mT5	50.40 / 65.84	48.97 / 64.71
Gon → Hi	Qwen	31.36 / 51.09	30.40 / 50.28
Gon → Hi	Llama	42.20 / 59.86	40.41 / 58.64

Table 5: Reference-model performance for Mundari. Each cell reports BLEU / ChrF++.

Direction	Model	Validation	Test
Hi → Mun	NLLB	21.41 / 47.95	20.39 / 46.65
Hi → Mun	mT5	26.85 / 51.77	26.53 / 51.62
Hi → Mun	Qwen	14.26 / 36.72	14.38 / 36.70
Hi → Mun	Llama	24.86 / 49.32	24.72 / 49.35
Mun → Hi	NLLB	50.98 / 65.81	50.88 / 65.67
Mun → Hi	mT5	57.65 / 71.38	57.44 / 71.14
Mun → Hi	Qwen	24.12 / 42.14	23.66 / 41.73
Mun → Hi	Llama	56.26 / 69.77	56.05 / 69.51

Table 6: Candidate-model validation performance (BLEU) template. This compact table establishes candidate diversity and reports the macro average used to justify the common mT5 SFT backbone. All values are illustrative placeholders.

Model	Hi→Bhi	Bhi→Hi	Hi→Gon	Gon→Hi	Hi→Mun	Mun→Hi	Macro
NLLB	31.4	38.2	29.8	40.7	30.6	39.4	35.0
mT5	34.8	42.7	35.1	45.6	34.9	43.8	39.5
Qwen	18.5	24.2	17.7	25.3	18.1	26.2	21.7
Llama	36.2	40.2	33.5	43.1	33.8	42.3	38.2

Table 7: Overall held-out test-performance template. RECAP-DPO is the primary result; GRPO and PPO are secondary refinement and optimizer baselines. All values are illustrative placeholders.

Method	BLEU $\uparrow$	ChrF++ $\uparrow$	COMET $\uparrow$	Rep. $\downarrow$	Length error $\downarrow$
SFT	34.82	57.31	0.612	4.82	0.143
DPO _{Ref}	35.67	58.04	0.621	4.71	0.139
RECAP-DPO	37.41	60.12	0.645	4.18	0.118
RECAP-DPO+GRPO	38.26	61.03	0.657	4.09	0.115
SFT $\rightarrow$ PPO	35.01	57.83	0.617	4.92	0.151

Table 8: Cumulative ablation of RECAP preference construction. This tests whether quality scoring, pathology penalties, margin filtering, and balanced all-pairs sampling each contribute. All values are illustrative placeholders.

Preference construction	BLEU $\uparrow$	ChrF++ $\uparrow$	COMET $\uparrow$	Retained pairs
Quality only (BLEU+ChrF++COMET)	35.72	58.23	0.623	1.00M
+ repetition penalty	36.31	59.01	0.631	1.00M
+ length penalty	36.08	58.72	0.629	1.00M
+ both penalties	36.74	59.46	0.636	1.00M
+ margin filtering	37.02	59.78	0.641	0.67M
Full RECAP	37.41	60.12	0.645	0.54M

Table 9: Controlled test of heterogeneous candidates and pair construction. Each condition should use matched final pair counts and comparable reward-margin distributions. All values are illustrative placeholders.

Preference source	Candidate pool	Pair strategy	BLEU $\uparrow$	ChrF++ $\uparrow$	COMET $\uparrow$
Self-sampling	mT5 only	Random	35.91	58.21	0.623
Heterogeneous	NLLB, mT5, Qwen, Llama	Best-worst	36.52	58.91	0.631
Heterogeneous	NLLB, mT5, Qwen, Llama	Random pairs	35.94	58.27	0.624
Heterogeneous	NLLB, mT5, Qwen, Llama	All pairs	37.08	59.71	0.641
Heterogeneous	NLLB, mT5, Qwen, Llama	RECAP balanced pairs	37.41	60.12	0.645

Table 10: Human validation of automatically constructed preferences. Evaluate a balanced subset of 500 candidate pairs across the six directions. All values are illustrative placeholders.

Preference construction	Agreement $\uparrow$	Human ρ $\uparrow$	Chosen wins $\uparrow$	Pairs
Quality only	71.2%	0.61	68.4%	500
+ repetition + length	75.8%	0.67	72.1%	500
+ calibration	78.4%	0.71	75.6%	500
Full RECAP	82.1%	0.77	79.8%	500

Table 11: Appendix reporting plan. These analyses support reproducibility and robustness but need not displace the five core main-paper tables.

Appendix item	Required content
Full direction results	BLEU, ChrF++, COMET, repetition, and length error for SFT, DPO _{Ref} , RECAP-DPO, and RECAP-DPO+GRPO in all six directions.
PPO comparison	Per-direction and macro-average SFT→PPO results against RECAP-DPO.
GRPO group size	Controlled comparison of $G \in \{2, 3\}$ on a fixed source subset.
GRPO data size	Controlled comparison of 10K, 25K, and 50K source subsets.
Preference statistics	Candidate comparisons, retained pairs, retention rate, mean margin, and source-level coverage by direction.
Pair composition	Counts and percentages for all six generator-pair types.
Seed variance	Per-seed scores and mean $\pm$ standard deviation for the main methods.